Assignment 1

PROBLEM STATEMENT:

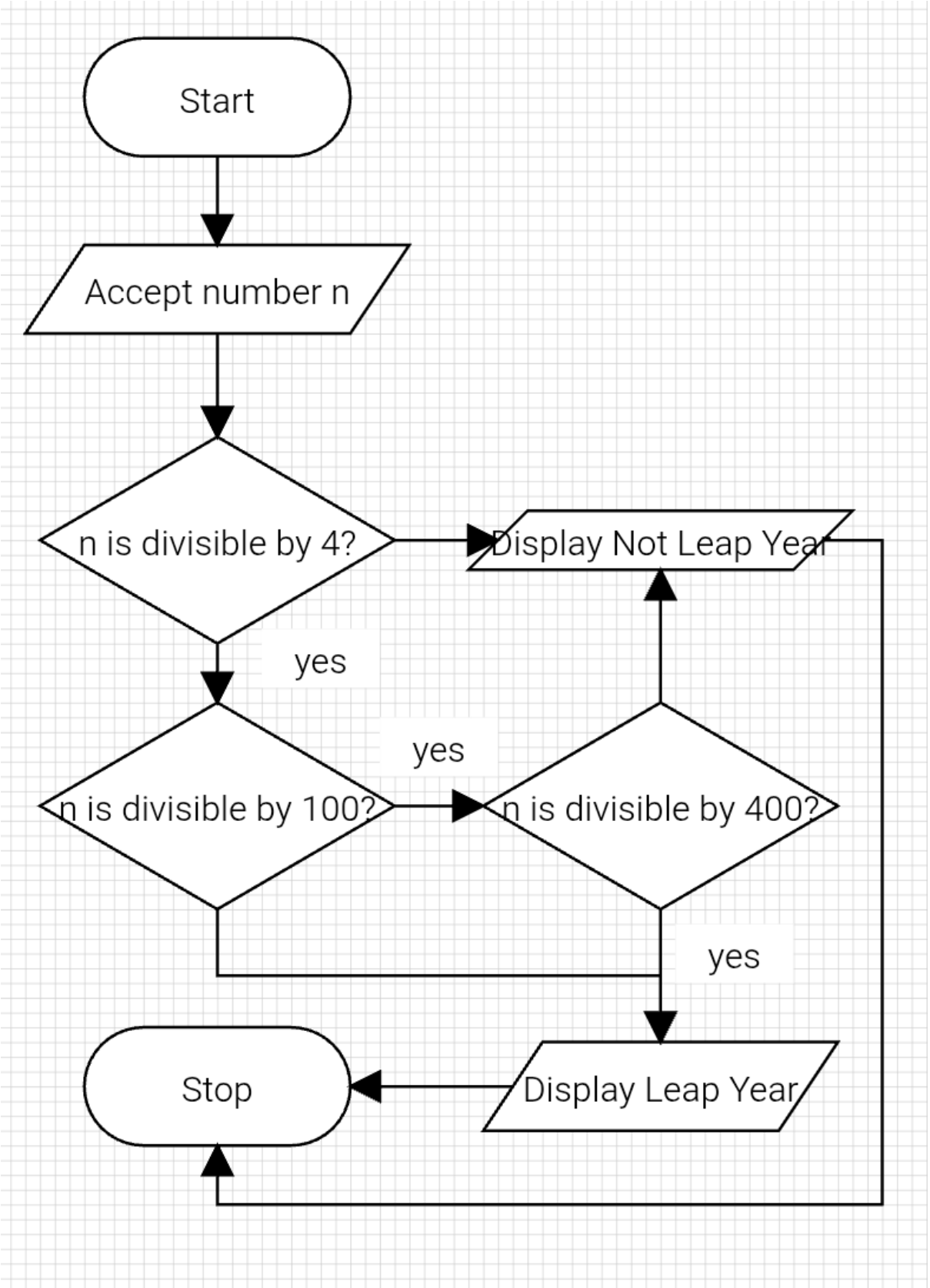
Write a program in C to check whether a given year is a Leap Year

OBJECTIVE:

To understand the use of conditional statements in C programming.
To implement logical conditions for solving real-world problems.
To develop problem-solving skills using decision control structures.

Algorithm:
Step 1: START
Step 2: Accept year n
Step 3: Check if n is divisible by 4 and if it is divisible by 100 and if it is divisble by 400
Step 4: If it is divisible by 4 but not by 100, or it is divisible by 400, display a leap year
Step 5: In any other case, display not a leap year
Step 6: STOP

FLOWCHART:



PLATFORM:

Linux

CODE:

```
#include <stdio.h>

void main(){
    int n;
    printf("Enter a Year - ");
    scanf("%d",&n);
    if ((n%4 == 0 && n%100 != 0) || (n%400 == 0)){
        printf("Leap Year");
    }
    else{
        printf("Not Leap Year");
    }
}
```

INPUT:

2024

OUTPUT:

```
Enter a Year - 2024
Leap Year
```

CONCLUSION:

Thus, the program successfully checks whether a given year is a Leap Year using conditional statements in C.

Assignment 2

PROBLEM STATEMENT:

Write a menu driven program in C to implement the basic arithmetic operations such as Addition, Subtraction, Multiplication, and Division.

OBJECTIVES:

• To understand the concept of menu driven programming in C.

• To apply conditional statements and switch–case control structures.

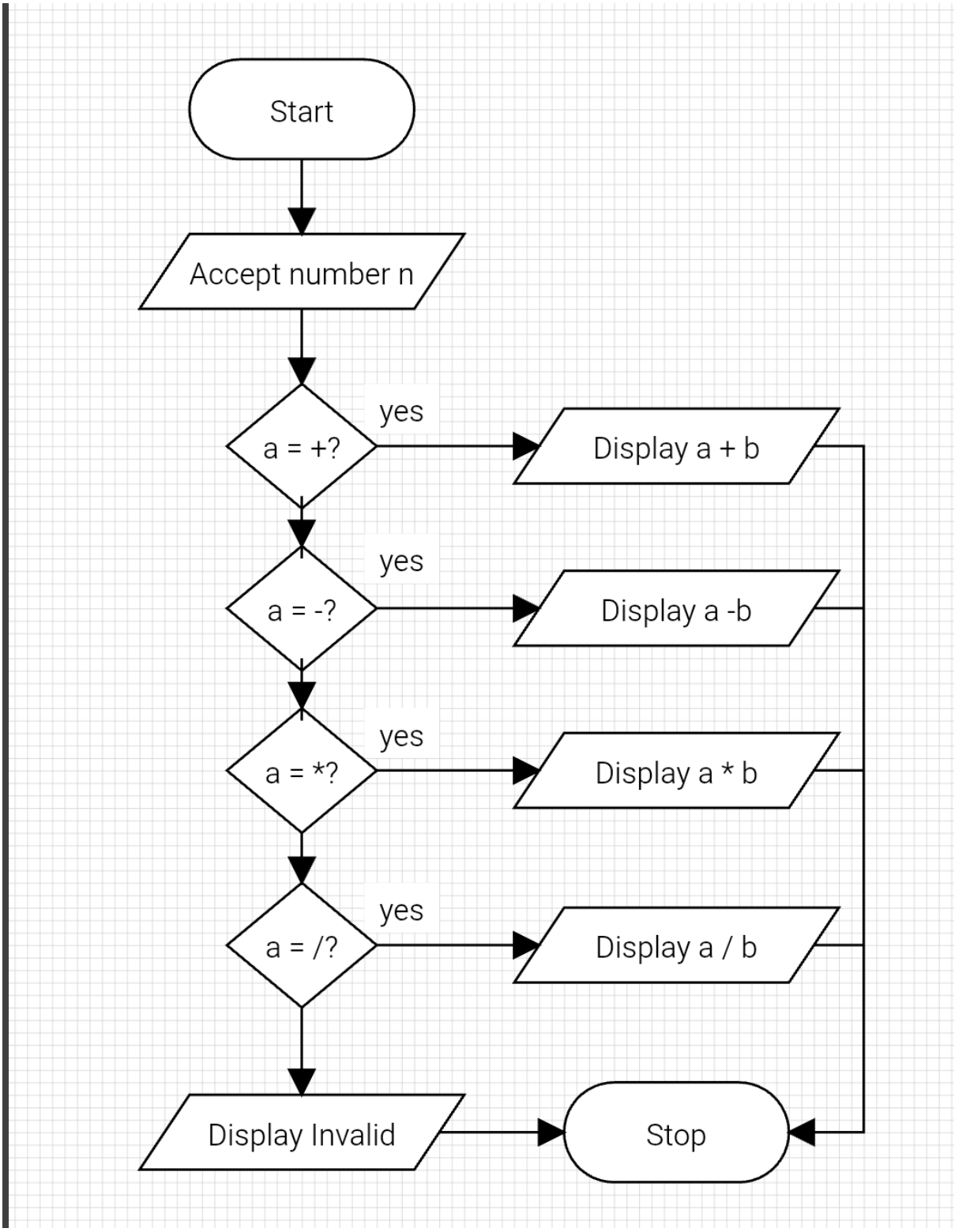
• To perform basic arithmetic operations using user-defined input.

• To enhance logical thinking and problem-solving skills.

ALGORITHM:

Step 1: START
Step 2: Accept operation input
Step 3: Accept two numbers a and b
Step 4: if input was +, display their sum
Step 5: if input was -, display their difference
Step 6: if input was *, display their product
Step 7: if input was /, display their quotient
Step 8: STOP

FLOWCHART:



PLATFORM:

Linux

CODE:

```
#include <stdio.h>
```

```
void main()
{
    char n;
    printf("Enter operation\n+ Addition\n- Subtraction\n* Multiplication\n/ Division\n");
    scanf("%c", &n);
    printf("Enter two numbers : ");
    float a,b;
    scanf("%f %f", &a, &b);
    switch(n){
        case '+':
            printf("%f", a+b);
            break;
        case '-':
            printf("%f", a-b);
            break;
        case '*':
            printf("%f", a*b);
            break;
        case '/':
            printf("%f", a/b);
            break;
        default:
            printf("Invalid Input");
    }
}
```

INPUT:

*

3 5

OUTPUT:

```
Enter operation
+ Addition
- Subtraction
* Multiplication
/ Division
*
Enter two numbers : 3 5
15.000000
```

CONCLUSION:

Thus, the menu driven program in C successfully performs basic arithmetic operations using switch–case statements.

Assignment 3

PROBLEM STATEMENT:

Write a program in C to generate multiplication tables for a given number.

OBJECTIVES:

- To understand the use of looping statements in C.
- To generate multiplication tables using loops.
- To improve program logic and execution flow understanding.

ALGORITHM:

Step 1: START

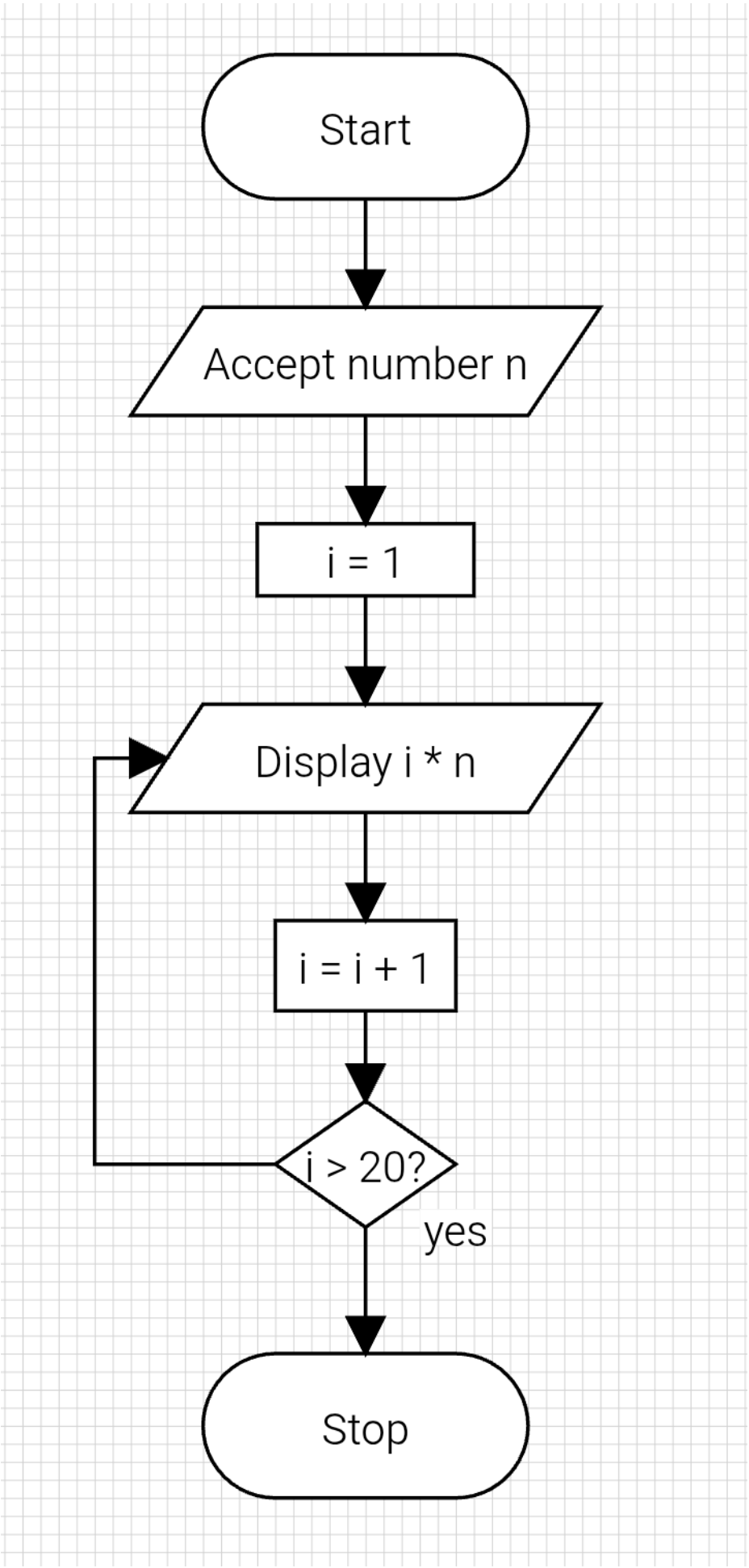
Step 2: Accept number n

Step 3: Iterate i from 1 to 20

Step 4: Each iteration, display i × n

Step 5: STOP

FLOWCHART:


Start
Accept number n
i = 1
Display i * n
i = i + 1
i > 20?
yes
Stop

PLATFORM:

Linux

CODE:

```
#include <stdio.h>


void main(){
    int n;
    printf("Enter number - ");
    scanf("%d",&n);
    for (int i = 1; i <=20; i++){
        printf("%d x %d = %d\n",n,i,n*i);
    }
}
```

INPUT:

5

OUTPUT:

```
Enter number - 5
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
5 x 6 = 30
5 x 7 = 35
5 x 8 = 40
5 x 9 = 45
5 x 10 = 50
5 x 11 = 55
5 x 12 = 60
5 x 13 = 65
5 x 14 = 70
5 x 15 = 75
5 x 16 = 80
5 x 17 = 85
5 x 18 = 90
5 x 19 = 95
5 x 20 = 100
```

CONCLUSION:

Thus, the menu driven program in C successfully performs basic arithmetic operations using switch–case statements.

Assignment 4

PROBLEM STATEMENT:

Write a C Program to calculate the salary of an employee given his basic pay (taken as input from the user). Calculate gross salary of employee. Let HRA be 10 % of basic pay and TA be 5% of basic pay. Let employees pay professional tax as 2% of total salary. Calculate net salary payable after deductions.

OBJECTIVE:

To understand the importance of flowchart for any programming model.

To learn simple flowchart symbols and arrows to define relationships.

To understand and develop visual representations of flow of data.

ALGORITHM:

Step 1: START

Step 2: Accept Basic Pay BP

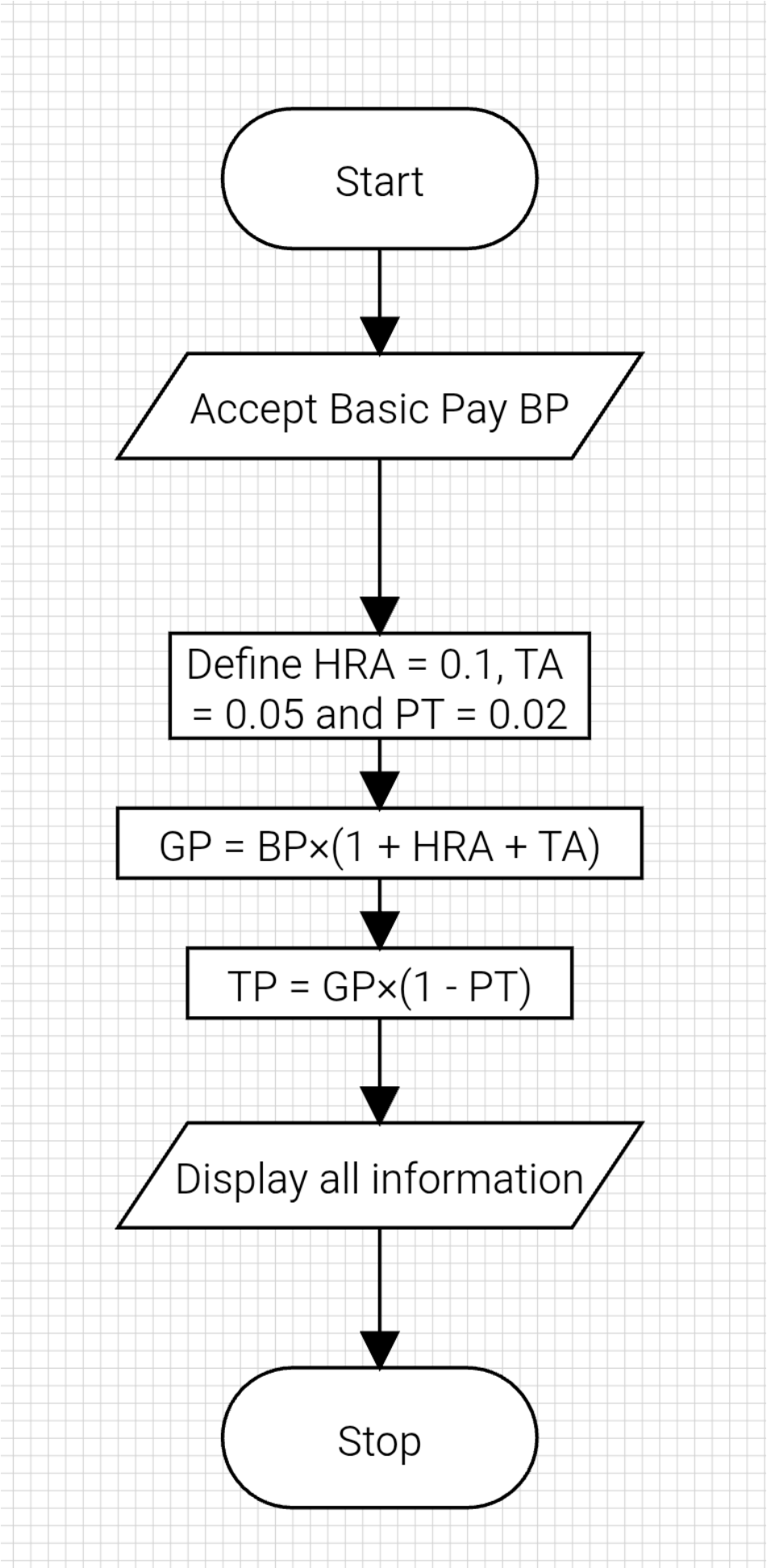
Step 3: Define HRA as 0.1, TA as 0.05 and PT as 0.02

Step 3: Calculate Gross salary using formula: BP×(1 - HRA - TA)

Step 4: Calculate Total Pay using GS×(1-PT)

Step 5: STOP

FLOWCHART:


Start
Accept Basic Pay BP
Define HRA = 0.1, TA = 0.05 and PT = 0.02
GP = BP×(1 + HRA + TA)
TP = GP×(1 - PT)
Display all information
Stop

PLATFORM:

Linux

CODE:

```
#include <stdio.h>


int main()
{
    printf("Enter Basic Pay - ");
    float BP;
    scanf("%f", &BP);
    float HRA = 0.1, TA = 0.05, PT = 0.02;
    float GS = BP*(1+HRA+TA);
    printf("Gross Salary - %f",GS * (1-PT));
}
```

INPUT:

1500

OUTPUT:

```
Enter Basic Pay - 1500
Gross Salary - 1690.500000
```

CONCLUSION: Thus, we have learned to write algorithms and to draw its representation in terms of flowchart.

Assignment 8

PROBLEM STATEMENT:
Write a program in C to accept marks of five courses of a student and compute the result. A student is considered PASS if he/she scores 40 marks or more in each course. If the student passes, calculate the aggregate percentage and assign the grade as follows:

- Aggregate ≥ 75% : Distinction
- Aggregate ≥ 60% and < 75% : First Division
- Aggregate ≥ 50% and < 60% : Second Division
- Aggregate ≥ 40% and < 50% : Third Division

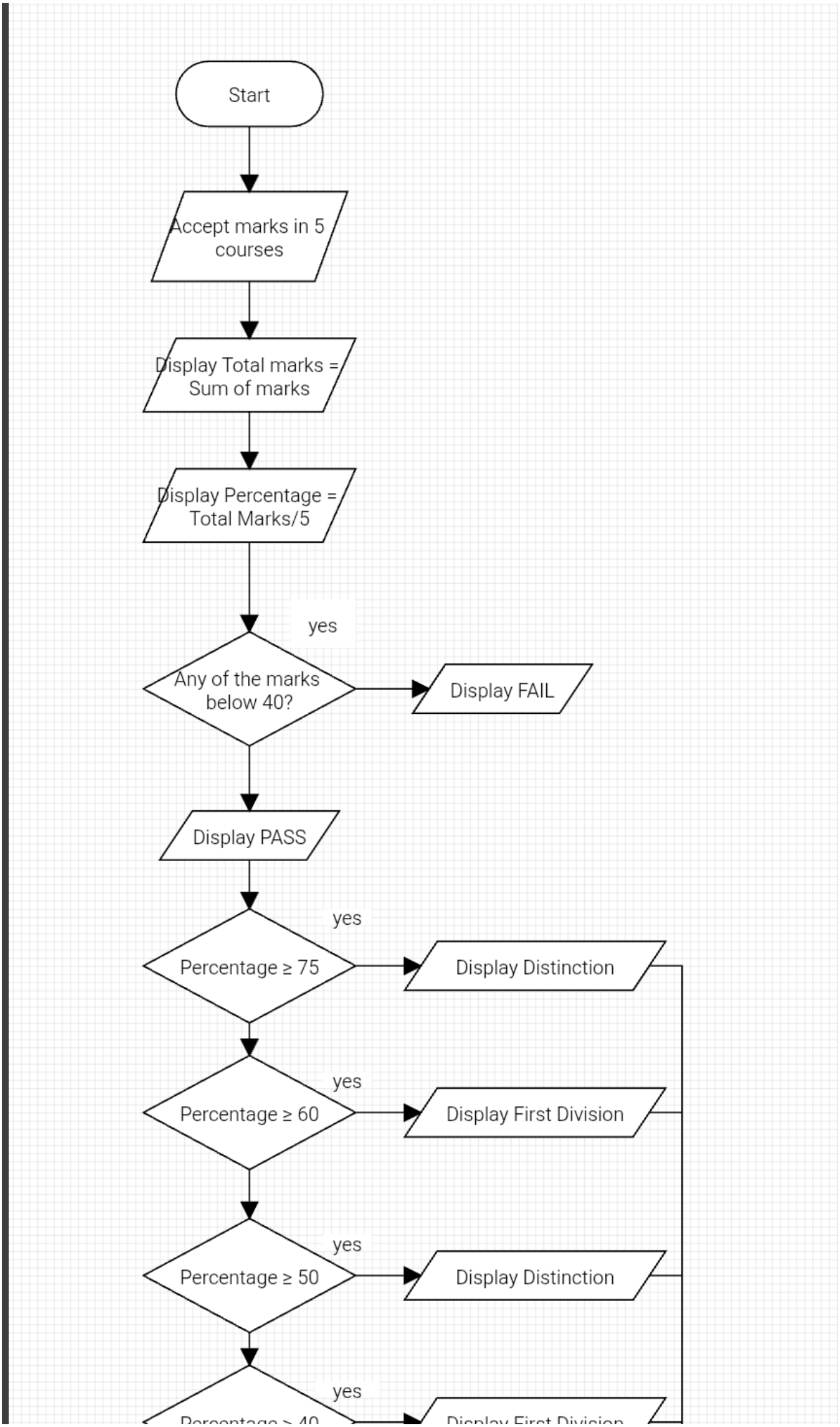
OBJECTIVE:

- To understand the use of conditional statements in C
- To apply logical operators for decision making
- To calculate total, percentage, and grade of a student
- To enhance problem-solving skills using real-life scenarios

THEORY:
In academic evaluation systems, student performance is assessed based on individual subject marks as well as overall aggregate performance. In C programming, such evaluation systems can be implemented using conditional statements like if, else if, and else. Logical operators help in checking multiple conditions simultaneously. Percentage calculation involves arithmetic operations, and grading logic is implemented based on predefined ranges. This program demonstrates structured programming and decision-making in C.

ALGORITHM:
Step 1: START
Step 2: Accept marks of 5 courses
Step 3: Define total marks as sum of all marks
Step 4: It any of the marks is below 40, display FAIL as the result
Step 5: Calculate Percentage as total marks divided by 5
Step 6: Define Grade as Distinction if percentage is over 75%, else First Division if percentage is over 60%, else Second Division if over 50% else Third Division if over 40%
Step 7: Display all information
Step 8: STOP

FLOWCHART:


Start
Accept marks in 5 courses
Display Total marks = Sum of marks
Display Percentage = Total Marks/5
yes
Any of the marks below 40?
Display FAIL
Display PASS
yes
Percentage ≥ 75
Display Distinction
yes
Percentage ≥ 60
Display First Division
yes
Percentage ≥ 50
Display Distinction
yes

PLATFORM:
Linux

CODE:

```
#include <stdio.h>

int main(){
   printf("Enter Marks in 5 Subjects : ");
   int m1,m2,m3,m4,m5;
   scanf("%d %d %d %d %d",&m1,&m2,&m3,&m4,&m5);
   int t = m1+m2+m3+m4+m5;
   float p = t / 5.0;
   int pass = 1;
   if (m1 < 40) pass = 0;
   if (m2 < 40) pass = 0;
   if (m3 < 40) pass = 0;
   if (m4 < 40) pass = 0;
   if (m5 < 40) pass = 0;
   printf("Total Marks = %d\nPercentage = %f%\nResult: %s",t,p,(pass?"PASS":"FAIL"));
   if (pass){
      printf("\nGrade: ");
      if (p >= 75) printf("Distinction");
      else if (p >= 60) printf("First Division");
      else if (p >= 50) printf("Second Division");
      else printf("Third Division");
   }
}
```

INPUT:
85 74 93 56 68

OUTPUT:

```
Enter Marks in 5 Subjects : 85 74 93 56 68
Total Marks = 376
Percentage = 75.199997%
Result: PASS
Grade: Distinction
```

CONCLUSION:
Thus, the program to calculate student result and grade based on marks was successfully implemented using C programming. This program demonstrates the effective use of conditional statements, logical operators, and arithmetic operations.

Assignment 9

PROBLEM STATEMENT:

Write a program in C to check whether the given number is an Armstrong number or not. An Armstrong number is an integer with three digits such that the sum of the cubes of its digits is equal to the number itself.
Example: 371

OBJECTIVE:

- To understand the concept of Armstrong numbers.
- To practice arithmetic operations in C.
- To develop logical thinking using loops and conditional statements.

THEORY:

An Armstrong number (also called a narcissistic number) is a number that is equal to the sum of the cubes of its digits. For a three-digit number abc, it is an Armstrong number if:
$a^3 + b^3 + c^3 = abc$
For example, 371 is an Armstrong number because:
$3^3 + 7^3 + 1^3 = 27 + 343 + 1 = 371$.

PLATFORM:

Linux

ALGORITHM:

Step 1: START

Step 2: Accept number n

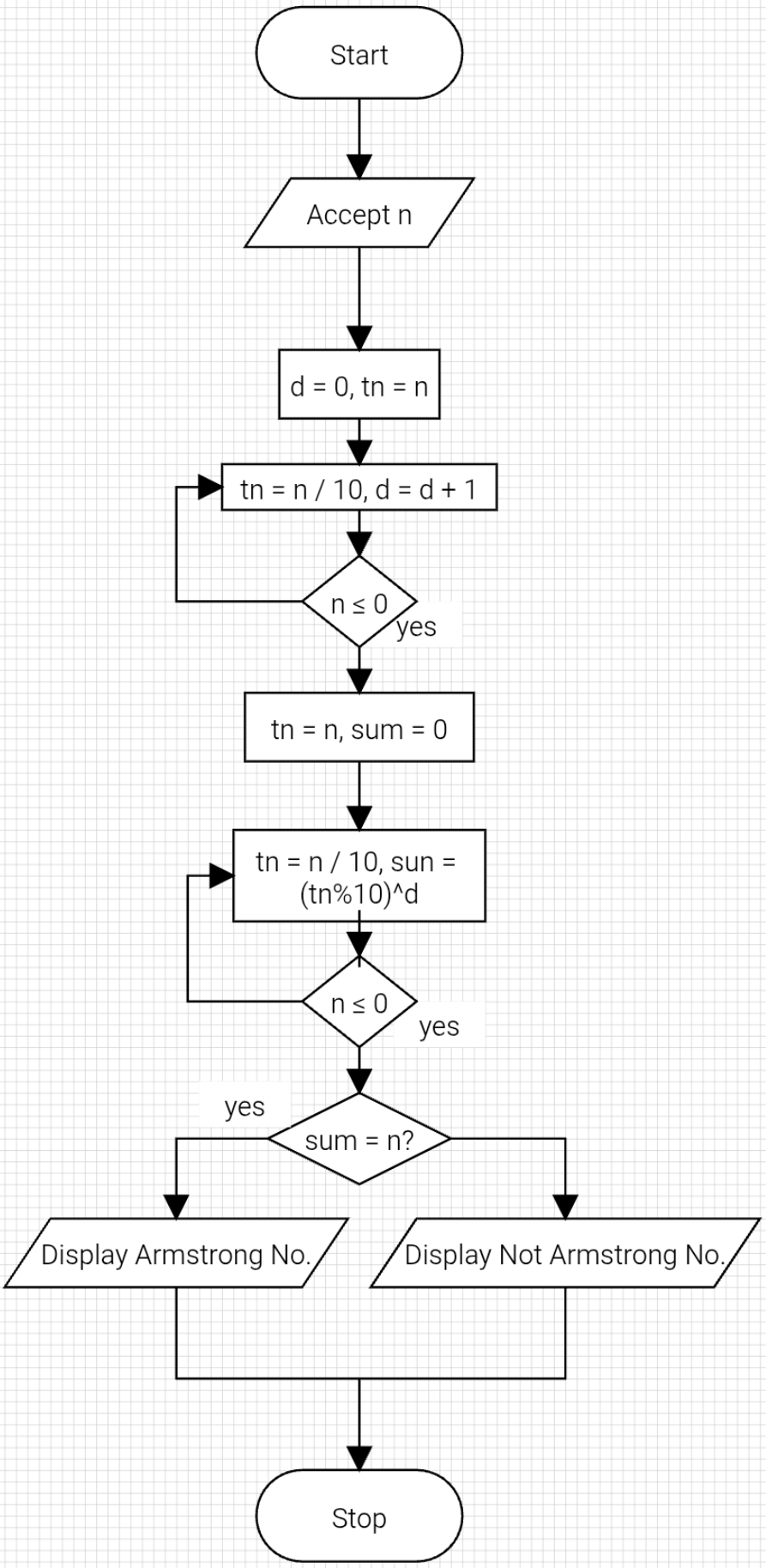
Step 3: Iterate through n to find digits d

Step 4: Iterate through n to find sum of all digits raised to d

Step 5: If sum equals n, display armstrong number else display not armstrong number

Step 6: STOP

FLOWCHART:


Start
Accept n
d = 0, tn = n
tn = n / 10, d = d + 1
n ≤ 0
yes
tn = n, sum = 0
tn = n / 10, sun = (tn%10)^d
n ≤ 0
yes
yes
sum = n?
Display Armstrong No.
Display Not Armstrong No.
Stop

CODE:

```
#include <stdio.h>
#include <math.h>

int isArmstrong(int n){
    int tn = n;
    int d = 0;
    while(tn > 0){
        tn /= 10;
        d++;
    }
    tn = n;
    int s = 0;
    while(tn > 0){
        s += pow(tn%10,d);
        tn /= 10;
    }
    if (s == n) return 1;
    return 0;
}

int main(){
    int n;
    printf("Enter number : ");
    scanf("%d",&n);
    printf(isArmstrong(n)?"Armstrong Number":"Not Armstrong Number");
}
```

INPUT:

153

OUTPUT:

```
Enter number : 153
Armstrong Number
```

CONCLUSION:

Thus, we have successfully written a C program to check whether a given number is an Armstrong number using loops and arithmetic operations.

Assignment 10

PROBLEM STATEMENT:

Write a program in C to simulate a simple calculator that performs basic arithmetic operations such as addition, subtraction, multiplication, and division. The calculator should also perform special operations like computing x^y (power) and x! (factorial).

OBJECTIVE:

- To understand the use of switch-case statements in C.
- To perform basic arithmetic and special mathematical operations.
- To enhance problem-solving skills using menu-driven programs.

THEORY:

A calculator is a basic computing device used to perform arithmetic operations. In C programming, a simple calculator can be implemented using conditional or switch-case statements. Arithmetic operations like addition, subtraction, multiplication, and division are fundamental operations. Special operations such as power (x^y) and factorial (x!) require loops for repeated calculations.

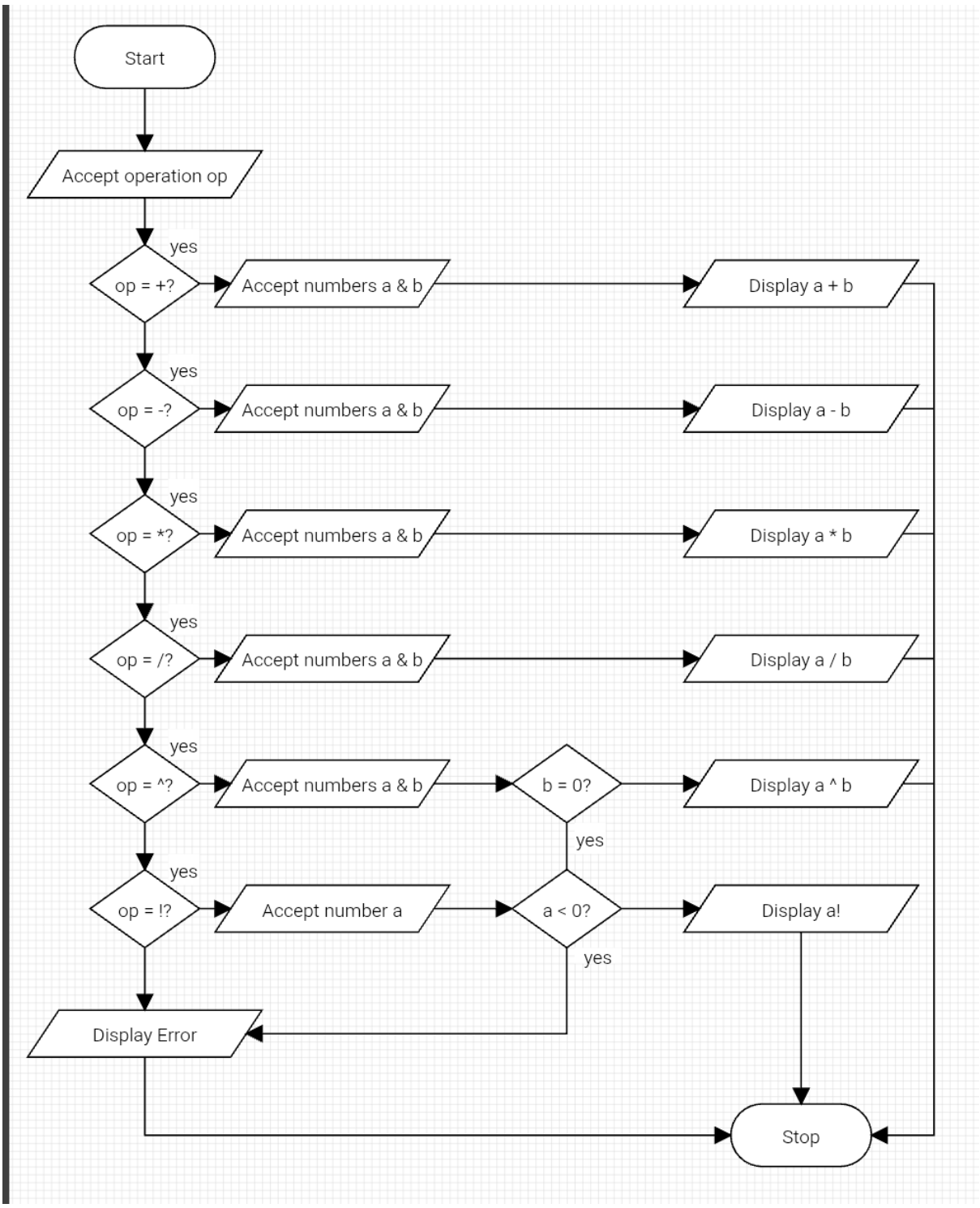
PLATFORM:

Linux

ALGORITHM:

Step 1: START
Step 2: Accept operation input
Step 3: Accept the required amount of numbers
Step 4: Check for illegal cases (division by zero, negative factorial)
Step 5: Perform given operation
Step 6: Display results
Step 7: STOP

FLOWCHART:



CODE:

```
#include<stdio.h>
#include<math.h>

long long fact(long long n){
    if (n == 1) return n;
    return (n * fact(n-1));
}
```

```
int main(){
    char op;
    printf("Enter Operation:\n+, -, *, /, ^, !\n");
    scanf("%c",&op);
    int a,b;
    if (op != '!'){
        printf("Enter 2 numbers : ");
        scanf("%d %d",&a,&b);
    }
    else{
        printf("Enter 1 number : ");
        scanf("%d",&a);
    }
    switch(op){
        case '+':
            printf("%d",a+b);
            break;
        case '-':
            printf("%d",a-b);
            break;
        case '*':
            printf("%d",a*b);
            break;
        case '/':
            printf("%d",a/b);
            break;
        case '^':
            printf("%f",pow(a,b));
            break;
        case '!':
            printf("%d",fact(a));
            break;
    }
}
```

INPUT:

^

5 3

OUTPUT:

```
Enter Operation:
+, -, *, /, ^, !
^
Enter 2 numbers : 5 3
125.000000
```

CONCLUSION:

Thus, a simple calculator program was successfully implemented using C programming. The program demonstrates the use of switch-case statements, loops, and arithmetic operations.

Assignment 11

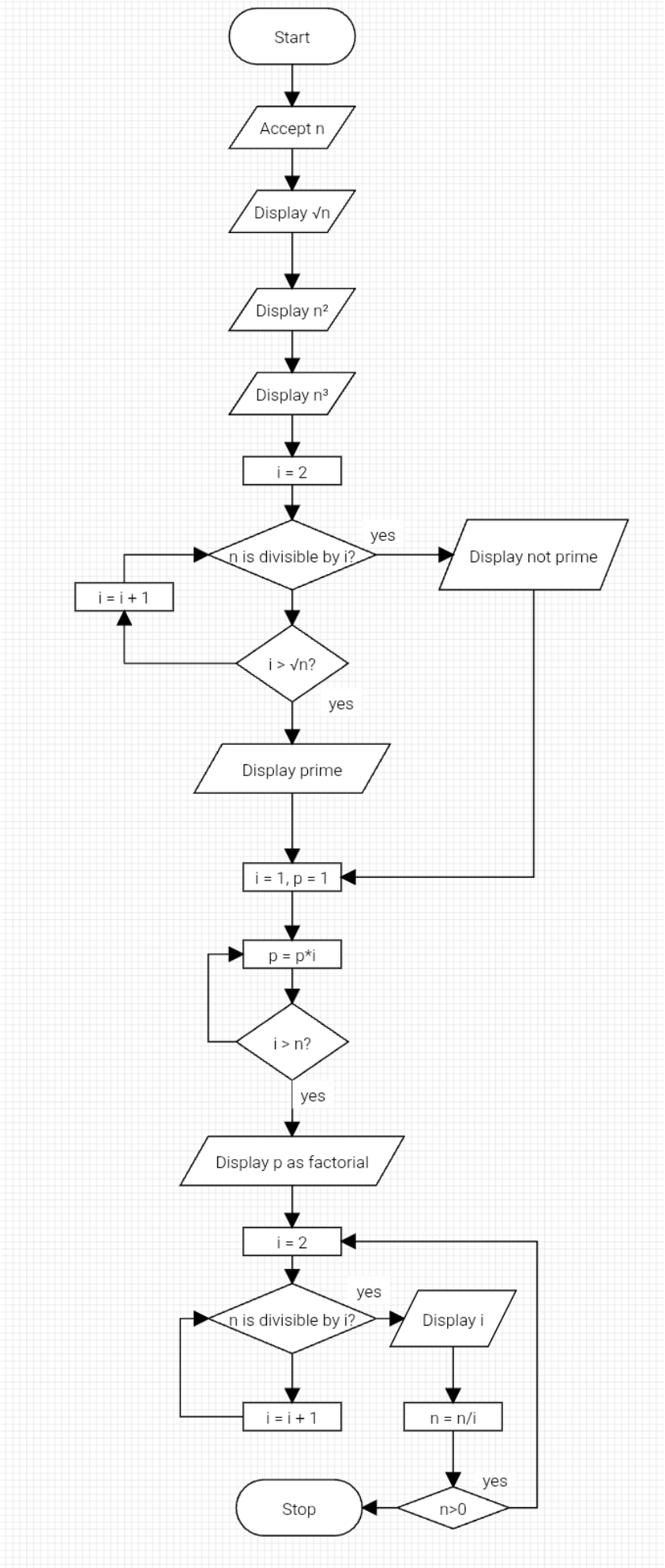
PROBLEM STATEMENT:
Write a program in C to accept a number from the user and compute the following:
a) Square root of the number
b) Square of the number
c) Cube of the number
d) Check whether the number is prime
e) Factorial of the number
f) Prime factors of the number

OBJECTIVE:
• To understand the use of mathematical operations in C.
• To implement loops and conditional statements
. • To perform number-based computations.
• To enhance logical thinking using menu-driven programs.

THEORY:
Numbers and their mathematical properties play a vital role in programming. Operations such as square, cube, and square root are basic mathematical computations. Prime number checking involves verifying whether a number has exactly two divisors. Factorial is calculated by multiplying a sequence of integers. Prime factorization breaks a number into its prime components. In C programming, these operations can be efficiently implemented using loops, conditional statements, and mathematical functions.

PLATFORM:
Linux
ALGORITHM:
Step 1: START
Step 2: Accept a number
Step 3: Display square root of number
Step 4: Display square of number
Step 5: Display Cube of number
Step 6: Display primality of number
Step 7: Display factorial of number
Step 8: Display prime factors of number
Step 9: STOP

FLOWCHART:


Start
Accept n
Display √n
Display n²
Display n³
i = 2
n is divisible by i?
yes
Display not prime
i = i + 1
i > √n?
yes
Display prime
i = 1, p = 1
p = p*i
i > n?
yes
Display p as factorial
i = 2
n is divisible by i?
yes
Display i
i = i + 1
n = n/i
yes
Stop
n>0

CODE:

```
#include <stdio.h>
#include <math.h>

int isPrime(int n){
    for (int i = 2; i <= sqrt(n); i++){
        if (n%i==0){
            return 0;
        }
    }
    return 1;
}

int fact(int n){
    if (n > 0){
        return n * fact(n-1);
    }
    return 1;
}

int main(){
    printf("Enter a number - ");
    int n;
    scanf("%d",&n);
    printf("Square Root - %f\nSquare of the number - %d\nCube of the number - %d\nIs
number Prime - %s\nFactorial of number - %d\nPrime Factors of the number -
",sqrt(n),n*n,n*n*n,(isPrime(n)?"Yes":"No"),fact(n));

    while (n > 1){
        for (int i = 2; i <= n; i++){
            if (n%i==0){
                printf("%d ", i);
                n/=i;
                break;
            }
        }
    }
}
```

INPUT:

12

OUTPUT:

```
Enter a number - 12
Square Root - 3.464102
Square of the number - 144
Cube of the number - 1728
Is number Prime - No
Factorial of number - 479001600
Prime Factors of the number - 2 2 3
```

CONCLUSION:

Thus, the program to perform various mathematical operations on a number was successfully implemented using C programming. This program demonstrates the use of loops, conditional statements, switch-case, and mathematical functions.

Assignment 12

PROBLEM STATEMENT:
Write a program in C to accept two numbers from the user and compute:
a) The smallest common divisor of the two numbers (other than 1, if any)
b) The Greatest Common Divisor (GCD) of the two numbers

OBJECTIVE:
• To understand the use of mathematical operations in C.
• To implement loops and conditional statements
. • To perform number-based computations.
• To enhance logical thinking using menu-driven programs.

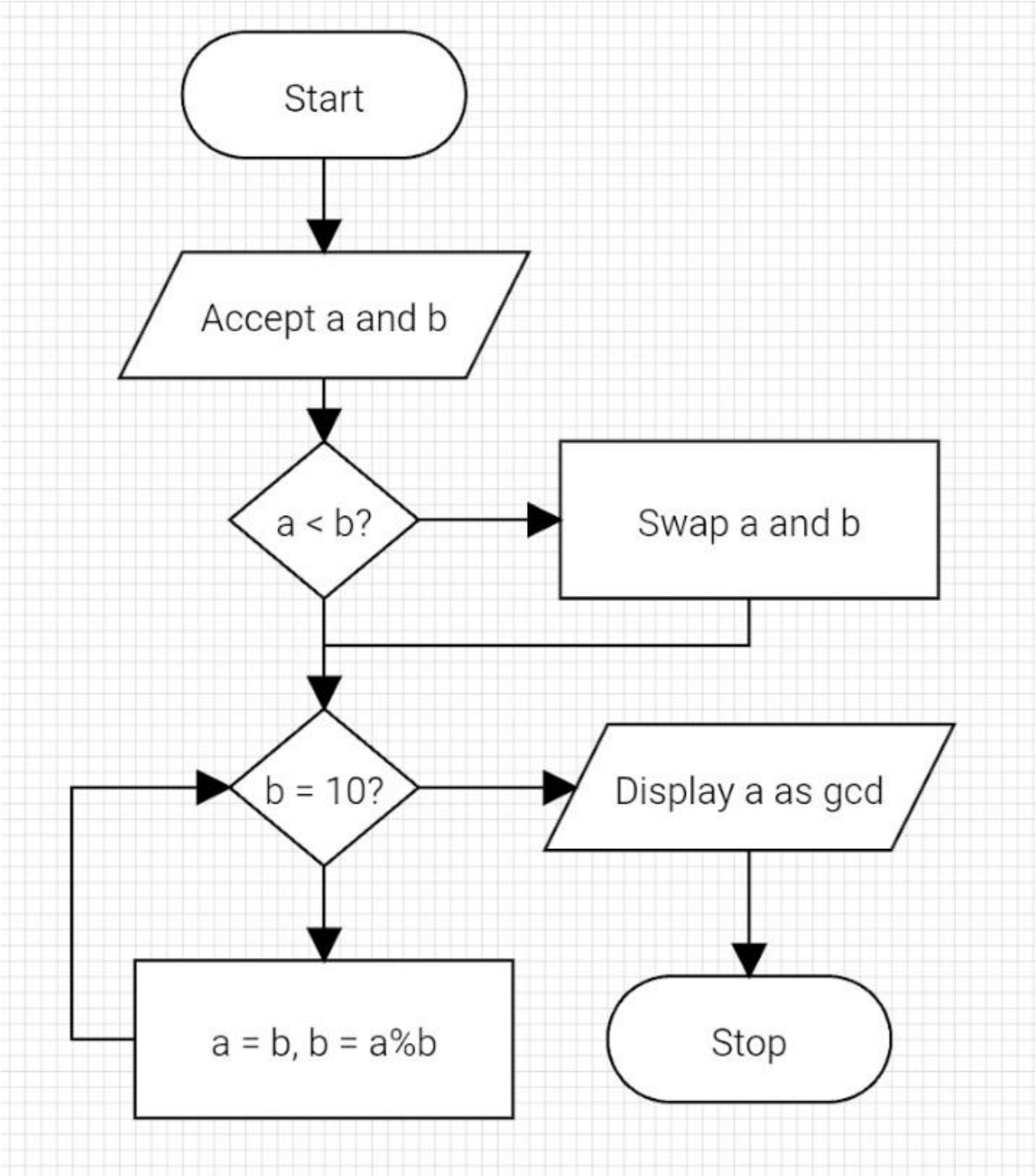
THEORY:
Divisors and the Greatest Common Divisor (GCD) are fundamental concepts in number theory. The smallest common divisor of two numbers is the smallest positive integer greater than 1 that divides both numbers. The GCD of two numbers is the largest positive integer that divides both numbers without leaving a remainder. In C programming, these operations can be efficiently implemented using loops and the Euclidean algorithm, which repeatedly applies the modulo operation until the remainder becomes zero.

PLATFORM:
Linux

ALGORITHM:

Step 1: START
Step 2: Accept numbers a & b (with b being the smaller number)
Step 3: If b is 0, GCD is a
Step 4: Set a as b and b as the remainder of a/b
Step 5: Repeat steps 3 & 4 until GCD is obtained
Step 6: Display GCD

FLOWCHART:



CODE:

```
#include <stdio.h>

int GCD(int a, int b){
    if (b == 0) return a;
    return(GCD(b, a%b));
}

int main() {
    printf("Enter 2 numbers - ");
    int a, b;
```

```
    scanf("%d %d",&a, &b);
    printf("GCD - %d", GCD(a,b));
}
```

INPUT:

27 63

OUTPUT:

```
Enter 2 numbers - 27 60
GCD - 3
```

CONCLUSION:

Thus, the program to find the smallest common divisor and the Greatest Common Divisor of two numbers was successfully implemented using C programming. This program demonstrates the use of loops, conditional statements, and the Euclidean algorithm.

Assignment 13

PROBLEM STATEMENT:
Write a program in C to accept a number from the user and print the digits of the number in reverse order.

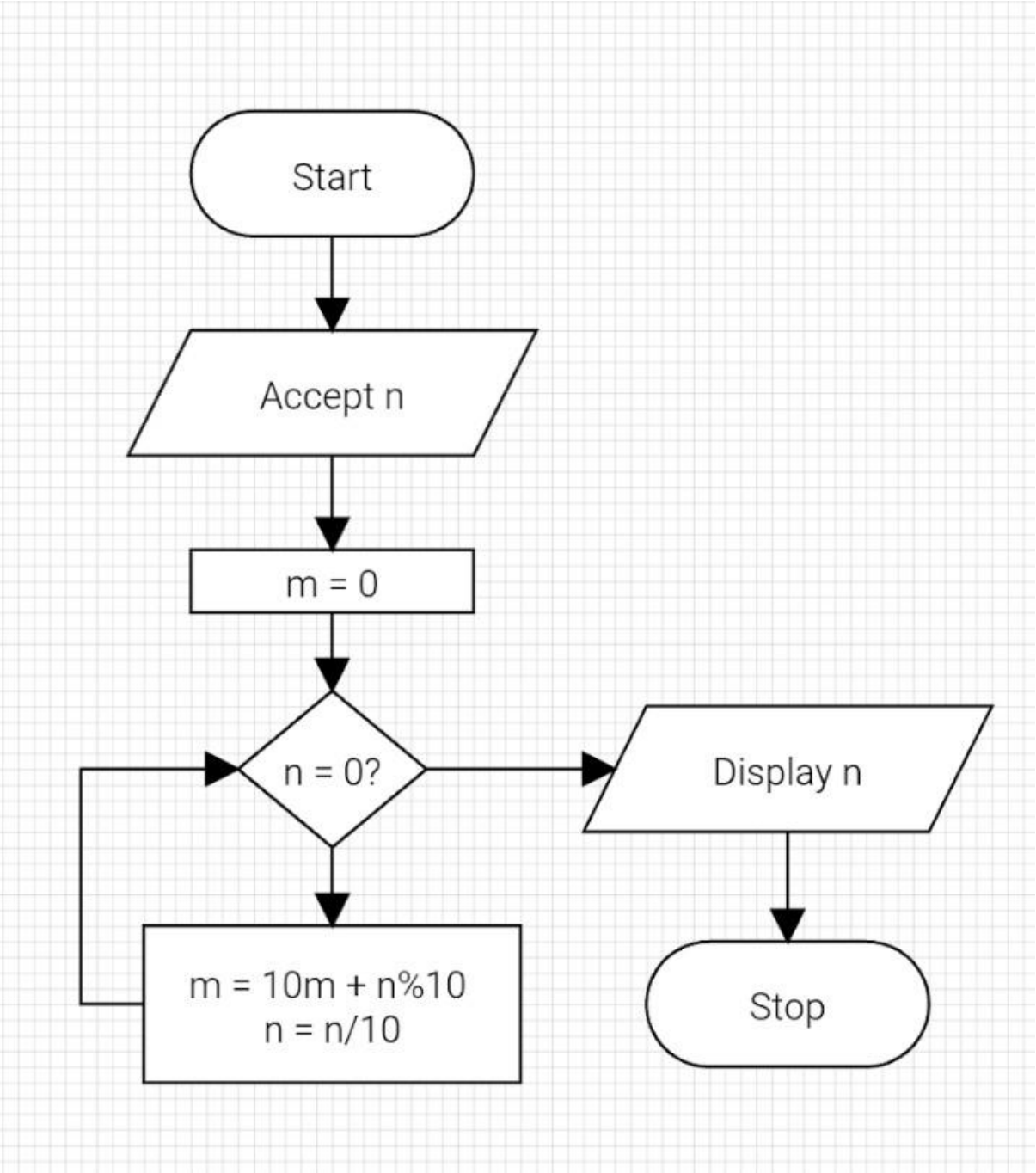
OBJECTIVE:
• To understand number manipulation using loops
. • To learn the use of modulo (%) and division (/) operators.
• To implement logic for reversing digits of a number.
• To improve problem-solving skills using iterative structures.

THEORY:
• To understand number manipulation using loops. • To learn the use of modulo (%) and division (/) operators. • To implement logic for reversing digits of a number. • To improve problem-solving skills using iterative structures.

PLATFORM:
Linux

ALGORITHM:
Step 1: START
Step 2: Accept number n
Step 3: Iterate through each digit of n starting from the unit digit while adding it to m multiplied by 10
Step 4: Display m
Step 5: STOP

FLOWCHART:



CODE:

```
#include <stdio.h>

int reverse(int n){
    int m = 0;
    while (n > 0){
        m = 10*(m) + n%10;
        n/=10;
    }
    return m;
}
```

```
int main() {
    printf("Enter a number - ");
    int n;
    scanf("%d",&n);
    printf("Reverse number - %d", reverse(n));
}
```

INPUT:
123
OUTPUT:

```
Enter a number - 123
Reverse number - 321
```

CONCLUSION:
Thus, the program to print the digits of a number in reverse order was successfully implemented using C programming. The program demonstrates the effective use of loops, modulo operator, and division operator.

Assignment 6

PROBLEM STATEMENT:
Write a C function to compute the factorial of a number with and without recursion.

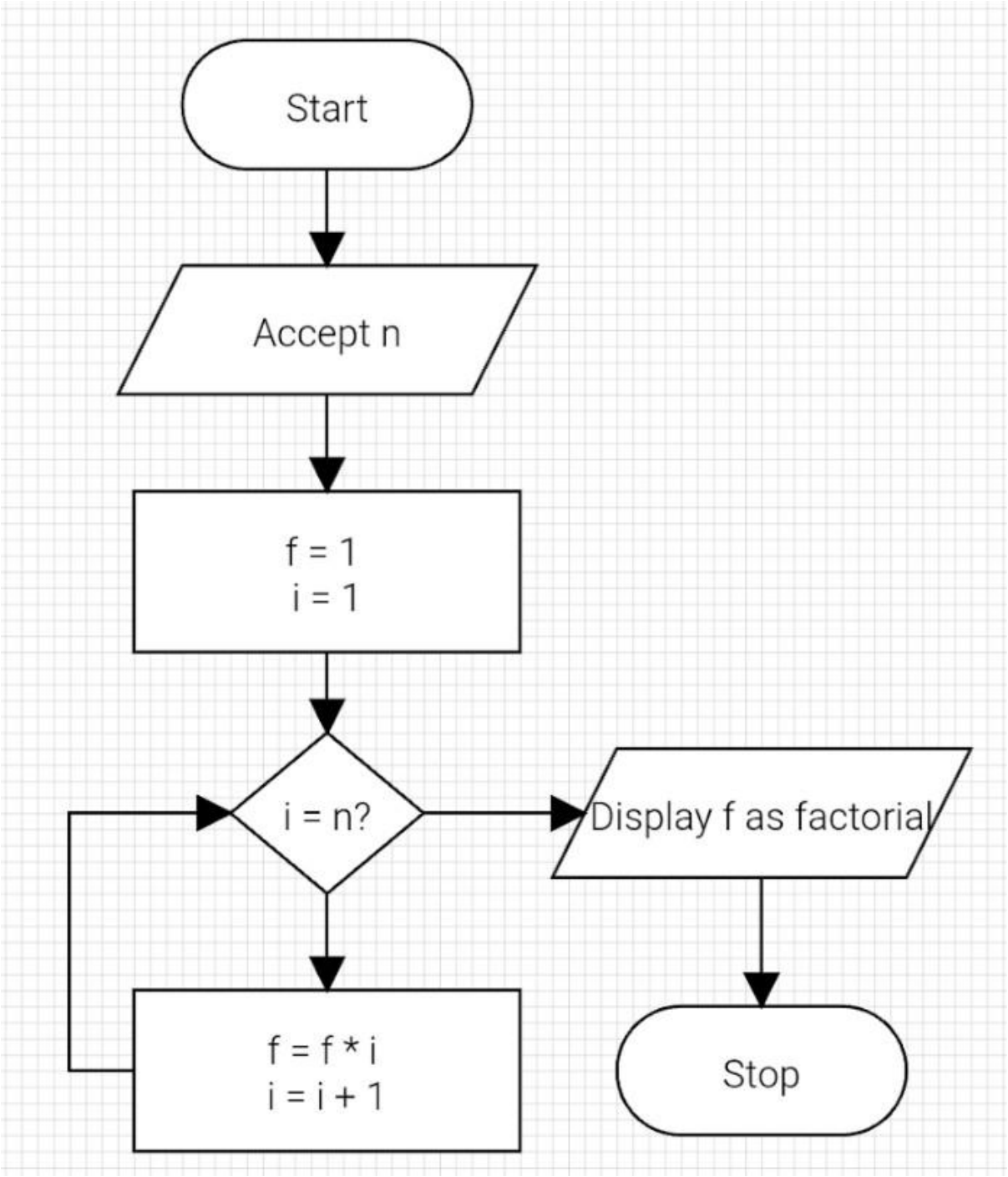
OBJECTIVE:
To demonstrate two different approaches for calculating the factorial of a number: iterative (non-recursive) and recursive, showcasing the versatility of problem-solving methods in Programming.

THEORY:
1) Explain the difference between the iterative and recursive approaches to calculating the factorial of a number.
2) What is the base case in the recursive function for computing factorials, and why is it necessary?
3) How does the program handle invalid input, such as negative numbers, and why is this Important?

PLATFORM:
Linux

ALGORITHM:
Step 1: START
Step 2: Accept number n
Step 3: Iterate i from 1 to n, and multiply a product variable defined originally as 1 by the loop count
Step 4: Display product as factorial
Step 5: STOP

FLOWCHART:



CODE:

```
#include <stdio.h>

int factrec(int n){
    if (n < 1) return 1;
    else return(n * factrec(n-1));
}

int factloop(int n){
    int f = 1;
```

```
    for (int i = 1; i <= n; i++){
        f *= i;
    }
    return f;
}

int main() {
    printf("Enter a number - ");
    int n;
    scanf("%d",&n);
    printf("Factorial with recursion - %d\nFactorial without recursion -
%d",factrec(n),factloop(n));
}
```

INPUT:
4

OUTPUT:

```
Enter a number - 4
Factorial with recursion - 24
Factorial without recursion - 24
```

CONCLUSION:
Thus, we have successfully executed factorial of a number with and without Recursion.

Assignment 7

PROBLEM STATEMENT:
Write a C program to accept student details and display their result
using an array of structures.

OBJECTIVE:

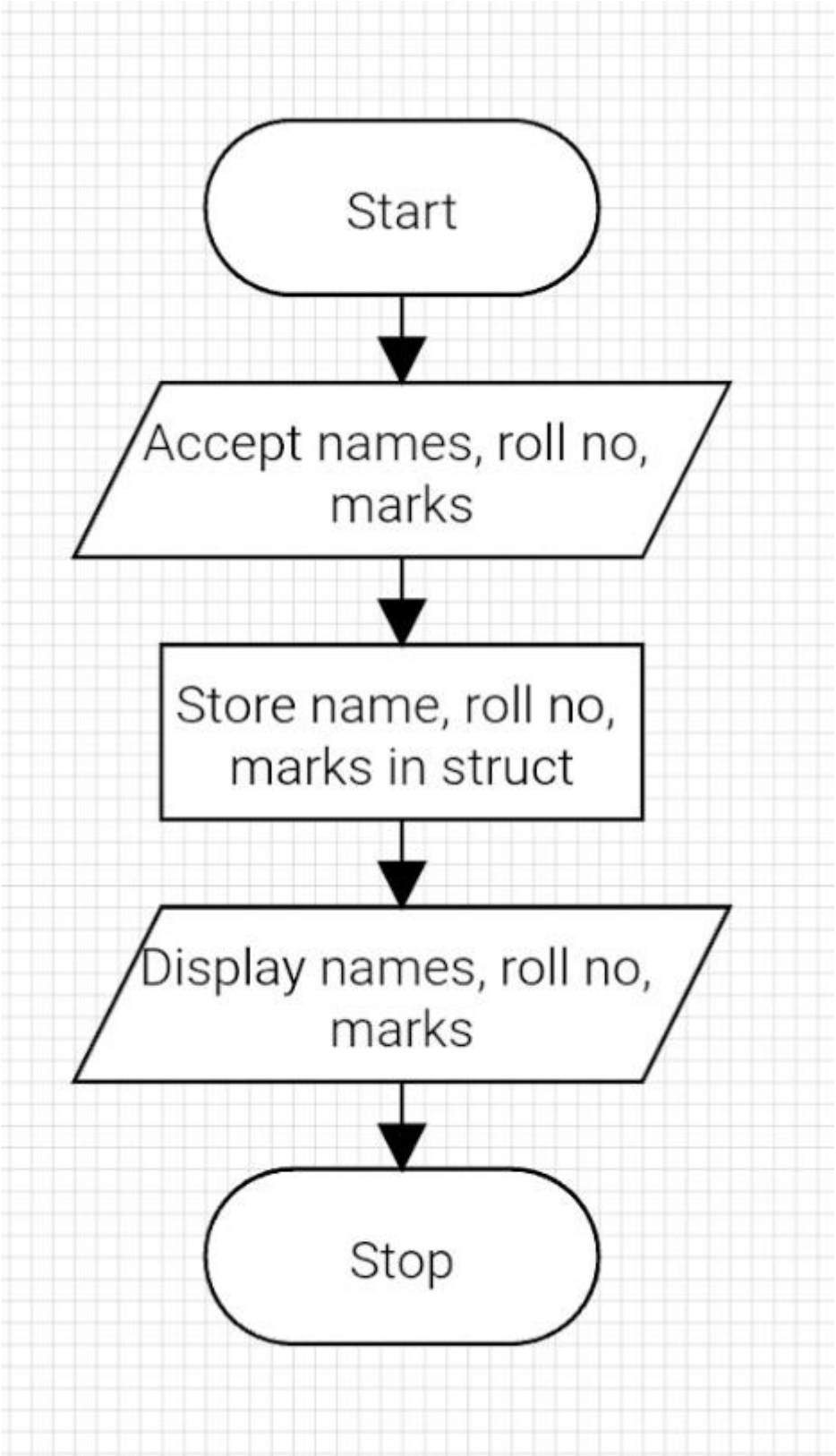
To understand the concept of structures in C.
To learn how to store multiple records using an array of structures.
To accept and process student details such as name, roll number, and marks.
To calculate total and percentage and display the result.

THEORY:
In C programming, a structure is a user-defined data type that allows grouping of variables of different data types under a single name. When multiple records such as student details need to be stored, an array of structures is used. This allows efficient storage and processing of student information like roll number, name, and marks in different subjects. The program can compute total marks and percentage and display the result for each student.

PLATFORM:
Linux

ALGORITHM:
Step 1: START
Step 2: Accept student name, roll no. and marks
Step 3: Store the accepted value in a struct
Step 4: Display details of student
Step 5: STOP

FLOWCHART:



CODE:

```
#include <stdio.h>
```

```
struct student{
    int rollno;
    int marks;
    char name[99];
};

int main() {
    printf("Enter Student name, marks and roll number\n");
    struct student s;
    fgets(s.name, 99 ,stdin);
    scanf("%d %d",&s.marks, &s.rollno);
    printf("Name - %sRoll Number - %d\nMarks - %d",s.name,s.rollno,s.marks);
}
```

INPUT:
Om Nayak
91
728

OUTPUT:

```
Enter Student name, marks and roll number
Om Nayak
91
728
Name - Om Nayak
Roll Number - 728
Marks - 91
```

.

CONCLUSION:
Thus, the C program successfully accepts student details using an array of structures and displays the calculated result including total marks and percentage.

Assignment 14

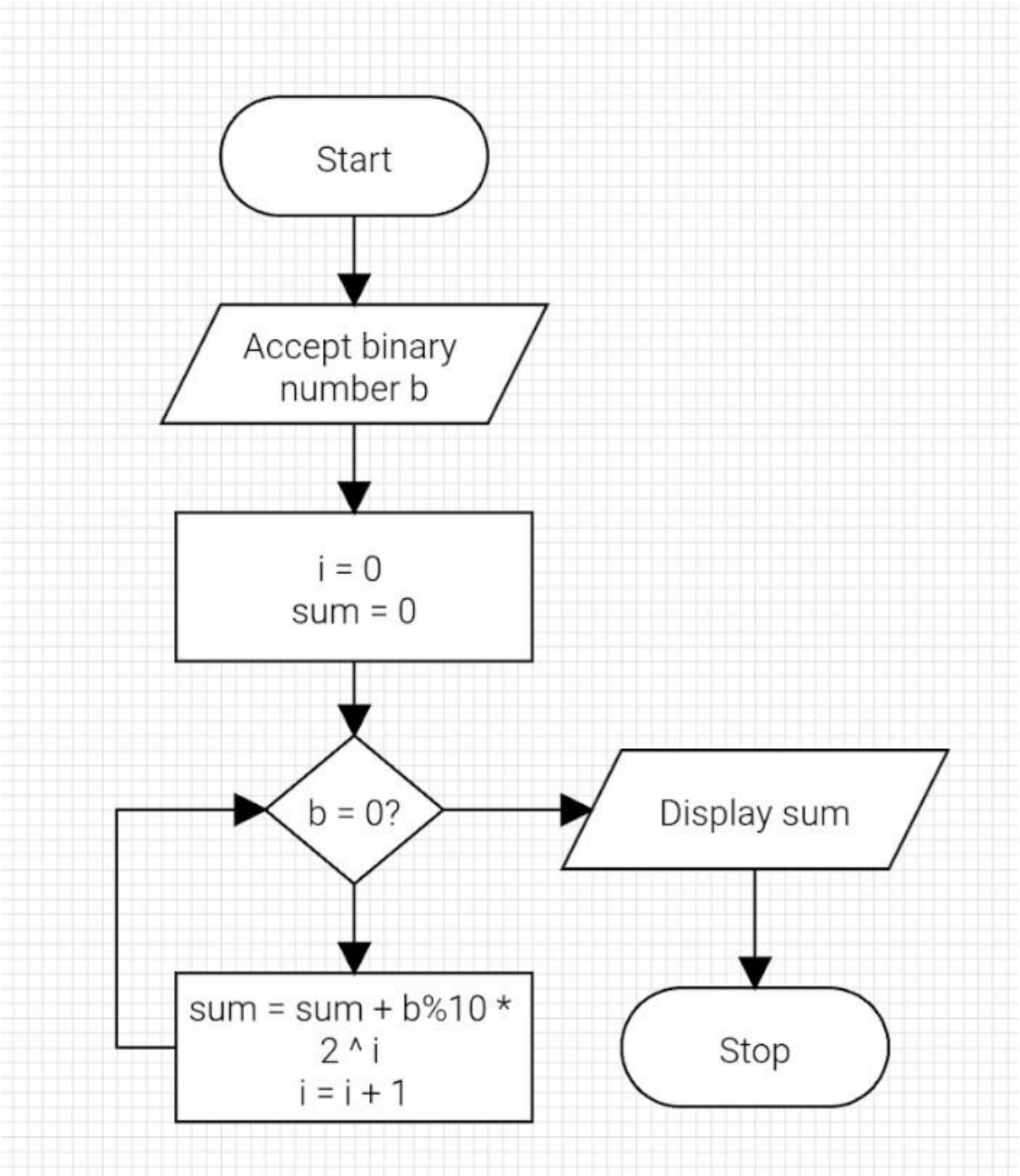
OBJECTIVE:
To understand number system conversions.
To learn how to convert binary numbers to decimal numbers in C.
To practice the use of loops and arithmetic operations.
To develop logical thinking for number system problems.

THEORY:
A binary number system uses only two digits: 0 and 1. Each digit of a binary number represents a power of 2 depending on its position. To convert a binary number into its decimal equivalent, each digit is multiplied by 2 raised to the power of its position and the results are added together. In C programming, this conversion can be implemented using loops and arithmetic operations.

PLATFORM:
Linux

ALGORITHM:
Step 1: START
Step 2: Accept binary number
Step 3: Iterate through binary number and add digit times 2^place value to a variable sum
Step 4: Display sum
Step 5: STOP

FLOWCHART:



CODE:

```
#include <stdio.h>
#include <math.h>


int bintodec(int b, int n){
    if (b == 0) return 0;
    return b%10 * pow(2,n) + bintodec(b/10,++n);
}

int main() {
```

```
    int n;
    printf("Enter Binary Number - ");
    scanf("%d", &n);
    printf("Decimal Number - %d",bintodec(n,0));
}
```

INPUT:
100101

OUTPUT:

```
Enter Binary Number - 100101
Decimal Number - 37
```

CONCLUSION:
Thus, the C program successfully converts a binary number entered by the user into its decimal equivalent using loops and arithmetic operations.

Assignment 15

PROBLEM STATEMENT:
Write a C program to generate pseudo random numbers.

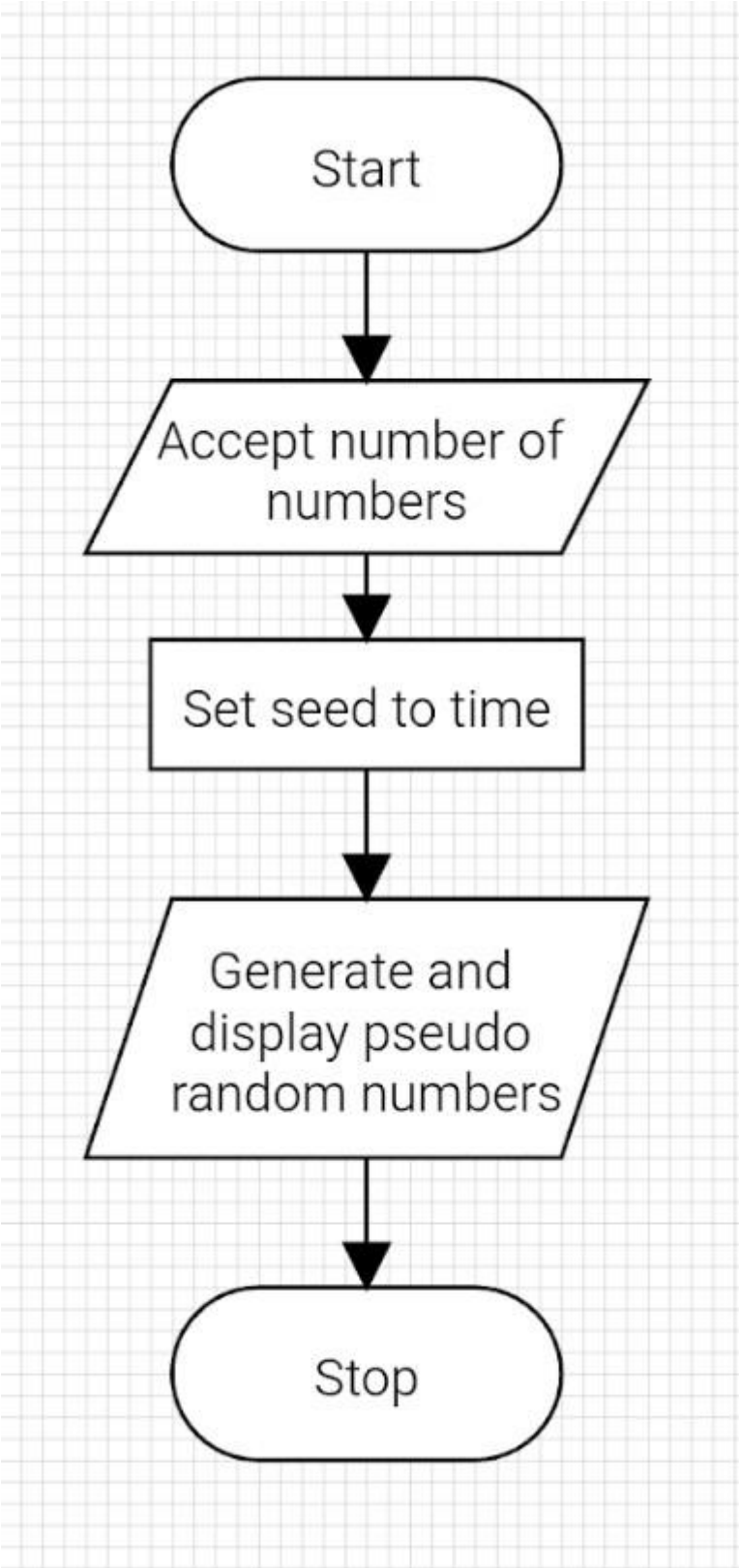
OBJECTIVE:
To understand the concept of pseudo random numbers.
To learn the use of rand() and srand() functions in C.
To generate random numbers using the standard library.
To understand the role of header files in C programming.

THEORY:
In C programming, pseudo random numbers are generated using the rand() function available in the stdlib.h header file. These numbers are not truly random but are generated using a deterministic algorithm. The srand() function is used to set the seed value for random number generation. Usually, the current time is used as a seed to produce different sequences of numbers each time the program runs.

PLATFORM:
Linux

ALGORITHM:
Step 1: START
Step 2: Accept no. of numbers to be generated n.
Step 3: Generate and display n pseudo-random numbers.
Step 4: STOP

FLOWCHART:



CODE:

```
#include  <stdio.h>
#include <stdlib.h>
#include <time.h>
```

```
int main() {
    srand(time(NULL));
    printf("Enter number of numbers - ");
    int n;
    scanf("%d",&n);
    for (int i = 0; i < n; i++){
        printf("%d\n",rand());
    }
}
```

INPUT:
5

OUTPUT:

```
Enter number of numbers - 5
288178756
742217379
868013132
1273272374
154862473
```

CONCLUSION: Thus, the C program successfully generates pseudo random numbers using the rand() function and srand() for seeding.

Assignment 16

PROBLEM STATEMENT:
To accept a list of N integers and partition the list into two sublists containing even and odd numbers.

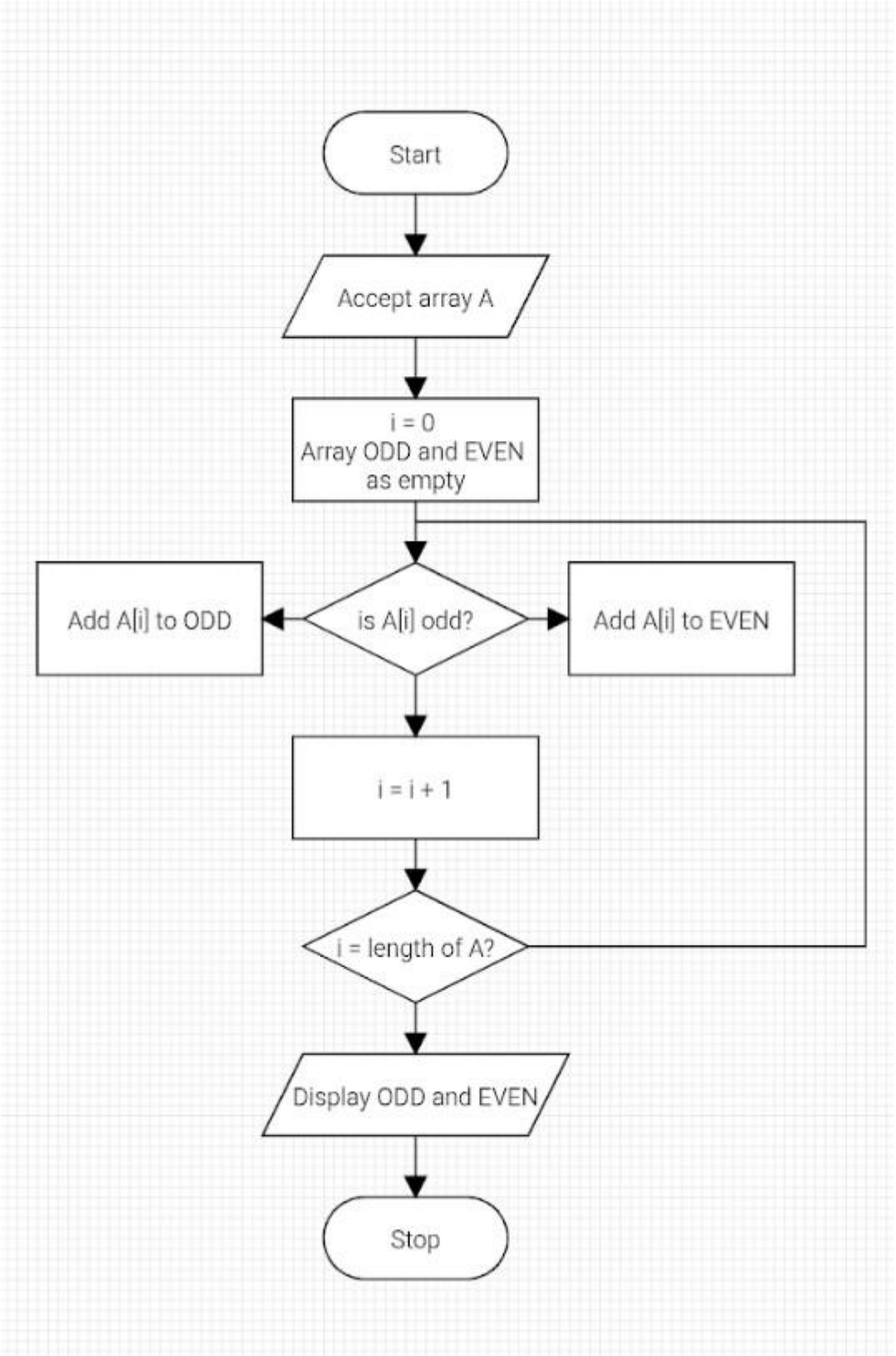
OBJECTIVE:
To understand how to work with arrays in C.
To learn how to separate numbers based on conditions.
To practice the use of loops and conditional statements.
To display two different lists for even and odd numbers.

THEORY:
In C programming, arrays are used to store multiple values of the same data type. By using loops and conditional statements, it is possible to examine each element in the array and classify it based on certain conditions. In this program, the user enters N integers which are stored in an array. Each number is then checked using the modulus operator (%). If a number is divisible by 2, it is categorized as an even number; otherwise, it is an odd number. The program then displays two separate lists: one for even numbers and another for odd numbers.

PLATFORM:
Linux

ALGORITHM:
Step 1: START
Step 2: Accept array of numbers A
Step 3: Create empty arrays called ODD and EVEN
Step 4: Iterate through A and add odd numbers to ODD and even numbers to EVEN.
Step 5: Display ODD and EVEN
Step 6: STOP

FLOWCHART:



CODE:

```
#include <stdio.h>

int main() {
```

```
    printf("Enter size of array - ");
    int n;
    scanf("%d",&n);
    int A[n];
    printf("Enter elements of array\n");
    for (int i = 0; i < n; i++){
        scanf("%d",&A[i]);
    }
    int ODD[n], EVEN[n];
    int oc = 0, ec = 0;
    for (int i = 0; i < n; i++){
        if (A[i] % 2 != 0) ODD[oc++] = A[i];
        else EVEN[ec++] = A[i];
    }
    printf("ODD - ");
    for (int i = 0; i < oc; i++){
        printf("%d ",ODD[i]);
    }
    printf("\nEVEN - ");
    for (int i = 0; i < ec; i++){
        printf("%d ",EVEN[i]);
    }
}
```

INPUT:
1 2 3 4 5

OUTPUT: .

```
Enter size of array - 5
Enter elements of array
1 2 3 4 5
ODD - 1 3 5
EVEN - 2 4
```

CONCLUSION:
Thus, the C program successfully accepts a list of integers and partitions them into two separate lists of even and odd numbers.

Assignment 5

PROBLEM STATEMENT:
Write a program in C to perform basic matrix operations such as:
1. Addition of two matrices
2. Saddle point of a matrix
3. Inverse of a matrix
4. Magic square of a matrix

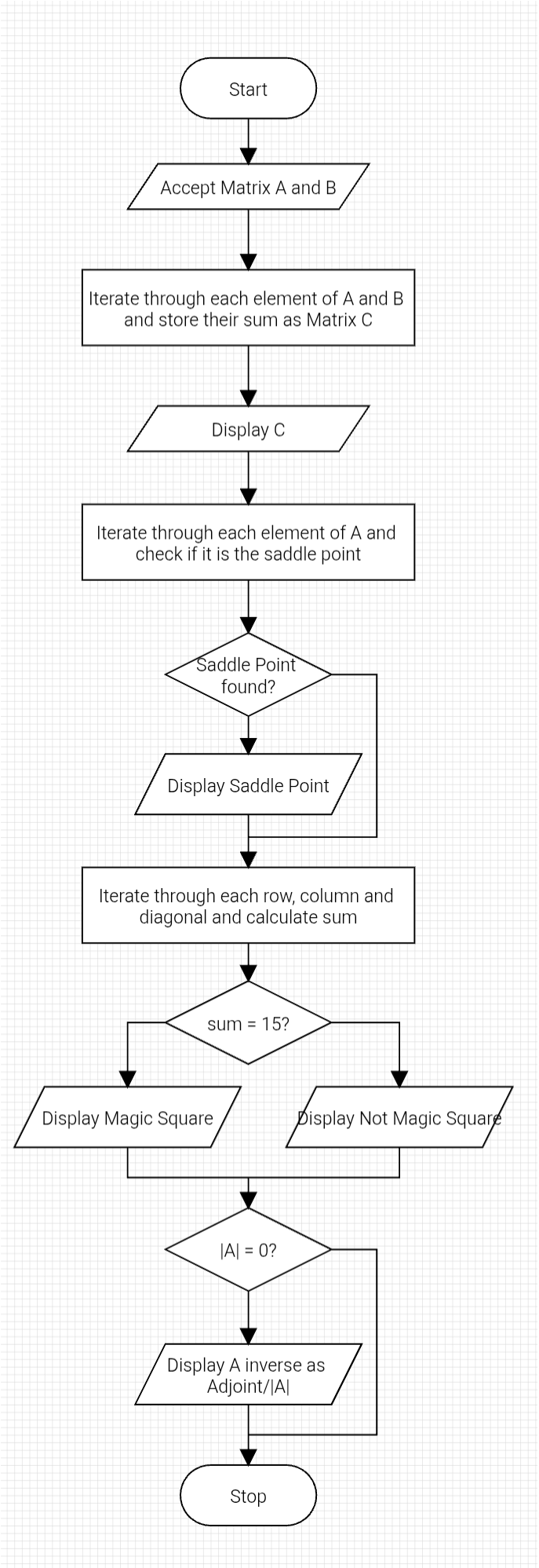
OBJECTIVE:
• To understand matrix representation in C using two-dimensional arrays.
• To implement various matrix operations using C programming.
• To develop logical thinking for solving matrix-based problems.

THEORY:
Matrix operations are widely used in engineering and scientific applications. Addition of matrices involves adding corresponding elements. A saddle point is an element that is the minimum in its row and maximum in its column. The inverse of a matrix exists only for square matrices with non-zero determinant. A magic square is a square matrix in which the sums of elements in each row, column, and diagonals are equal.

PLATFORM:
Linux

ALGORITHM:
1. Start
2. Read the order of the matrix
3. Read elements of the matrices
4. Perform matrix addition
5. Check for saddle point
6. Find inverse of the matrix (if determinant ≠ 0)
7. Check whether the matrix is a magic square
8. Display results
9. Stop

FLOWCHART:


Start
Accept Matrix A and B
Iterate through each element of A and B and store their sum as Matrix C
Display C
Iterate through each element of A and check if it is the saddle point
Saddle Point found?
Display Saddle Point
Iterate through each row, column and diagonal and calculate sum
sum = 15?
Display Magic Square
Display Not Magic Square
|A| = 0?
Display A inverse as Adjoint/|A|
Stop

CODE:

```
#include <stdio.h>
#include <math.h>

void cofactor(int n, int a[n][n], int c[n-1][n-1], int ai, int aj);

void printm(int n, int a[n][n]){
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++){
            printf("%d ",a[i][j]);
        }
        printf("\n");
    }
}

void printfm(int n, float a[n][n]){
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++){
            printf("%f ",a[i][j]);
        }
        printf("\n");
    }
}

int Determinant(int n, int a[n][n])
{
    int d = 0;
    if (n == 2)
    {
        d = a[0][0]*a[1][1] - a[0][1]*a[1][0];
        return (d);
    }
    int c[n-1][n-1];
    for (int i = 0; i < n; i++){
        cofactor(n,a,c,i,0);
        d += pow(-1,i) * a[i][0] * Determinant(n-1,c);
    }
    return d;
}

void cofactor(int n, int a[n][n], int c[n-1][n-1], int ai, int aj)
{
    int ioff = 0;
    for (int i = 0; i < n-1; i++)
    {
```

```
        int joff = 0;
        if (i == ai){
            ioff = 1;
        }
        for (int j = 0; j < n-1; j++){
            if (j == aj){
                joff = 1;
            }
            c[i][j] = a[i + ioff][j +joff];
        }
    }
}

void adjoint(int n, int a[n][n], float b[n][n]){
    for (int i = 0; i < n; i++){
        for (int j = 0; j < n; j++){
            int c[n-1][n-1];
            cofactor(n,a,c,i,j);
            b[i][j] = Determinant(n-1,c);
        }
    }
}

void inverse(int n, int a[n][n], float b[n][n]){
    int d = Determinant(n,a);
    adjoint(n,a,b);
    for (int i = 0; i < n; i++){
        for (int j = 0; j < n; j++){
            b[n][n] = b[n][n] * d;
        }
    }
}

void inputmatrix(int n, int a[n][n]){
    for (int i = 0; i < n; i++){
        for (int j = 0; j < n; j++)
        {
            scanf("%d",&a[i][j]);
        }
    }
}

void addmatrices(int n, int a[n][n], int b[n][n], int c[n][n]){
    for (int i = 0; i < n; i++){
        for (int j = 0; j < n; j++)
        {
            c[i][j] = a[i][j] + b[i][j];
        }
```

```
    }
}

int saddlepoint(int n, int a[n][n], int *check){
    for (int i = 0; i < n; i++){
        for (int j = 0; j < n; j++){
            for (int k = 0; k < n; k++){
                if (a[i][j] > a[i][k]) goto endofloop;
            }
            for (int k = 0; k < n; k++){
                if (a[i][j] < a[k][j]) goto endofloop;
            }
            *check = 1;
            return a[i][j];
            endofloop:
        }
    }
    return 0;
}

int ismagicsquare(int n, int a[n][n]){
    for (int i = 0; i < n; i++){
        for (int j = 0; j < n; j++){
            for (int k = 0; k < n; k++){
                for (int l = 0; l < n; l++){
                    if (a[i][j] == a[k][l] && i != k && j != l){
                        return 0;
                    }
                }
            }
        }
        int sum = 0;
        for (int j = 0; j < n; j++){
            sum += a[i][j];
        }
        if (sum!= 15) return 0;
        sum = 0;
        for (int j = 0; j < n; j++){
            sum += a[j][i];
        }
        if (sum!= 15) return 0;
        sum = 0;
        for (int j = 0; j < n; j++){
            sum += a[j][j];
        }
        if (sum!= 15) return 0;
    }
    return 1;
```

```
}

int main() {
    int n;
    printf("Enter Size of Matrices\n");
    scanf("%d",&n);
    printf("Enter Elements of Matrix A\n");
    int a[n][n];
    inputmatrix(n,a);
    printf("Enter Elements of Matrix B\n");
    int b[n][n];
    inputmatrix(n,b);
    printf("\n\nSum of Matrix A and B:\n");
    int sum[n][n];
    addmatrices(n,a,b,sum);
    printm(n,sum);
    printf("\n");
    int saddlepointexists = 0;
    int saddlepointn = saddlepoint(n, a, &saddlepointexists);
    if (saddlepointexists) printf("Saddle Point of A:\n%d",saddlepointn);
    printf("A is %s Magic Square",ismagicsquare(n,a)?"a":"not a");
    float inv[n][n];
    if (Determinant(n,a) != 0){
    inverse(n,a,inv);
    printf("\n\nInverse of A:\n");
    printfm(n,inv);
    }
}
```

INPUT:

1 0 -2
3 1 -2
-5 -1 9

&

1 0 1
0 1 0
1 0 1

OUTPUT:

```
Enter Size of Matrices
3
Enter Elements of Matrix A
1 0 -2
3 1 -2
-5 -1 9
Enter Elements of Matrix B
1 0 1
0 1 0
1 0 1


Sum of Matrix A and B:
2 0 -1
3 2 -2
-4 -1 10

A is not a Magic Square

Inverse of A:
7.000000 17.000000 2.000000
-2.000000 -1.000000 -1.000000
2.000000 4.000000 1.000000
```

CONCLUSION:

Thus, we have learned how to perform basic matrix operations using C programming and understood their practical applications.

Assignment 17

PROBLEM STATEMENT:
To accept the number of terms and find the sum of the sine series.

OBJECTIVE:
- To understand the concept of mathematical series in programming.
- To implement sine series using C programming.
- To practice loops and arithmetic operations.
- To learn the use of factorial and power calculations in C.

THEORY:
In mathematics, the sine function can be expressed as an infinite series called the Taylor series expansion. The sine series is represented as:
sin(x) = x − x^3/3! + x^5/5! − x^7/7! + ...

In this program, the user enters the value of x and the number of terms. The program calculates the sum of the sine series up to the specified number of terms using loops. Factorial and power operations are used to compute each term of the series.
PLATFORM:
Linux

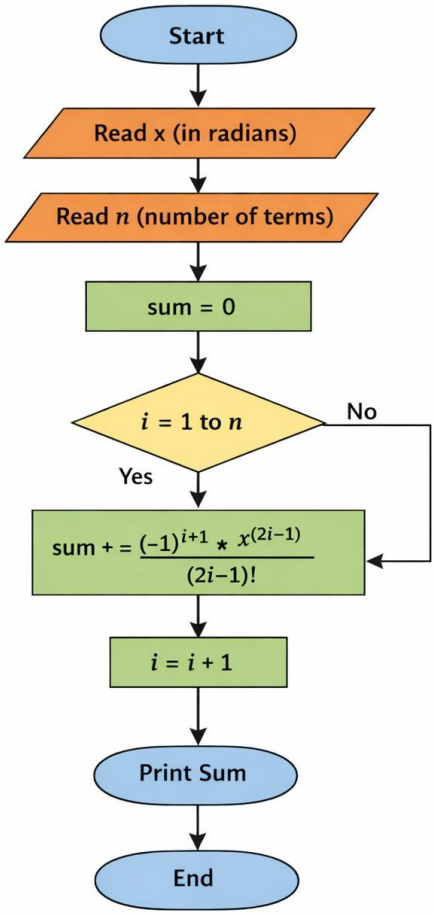
CODE:
```
#include <stdio.h>
#include <math.h>

float fact(int n){
   if (n > 0){
      return n * fact(n-1);
   }
   return 1;
}

int main() {
   float x;
   int n;
   printf("Enter x (in radians) : ");
   scanf("%f",&x);
   printf("Enter number of terms : ");
   scanf("%d",&n);
   float sum = 0;
   for (int i = 1; i <= n; i++){
      sum += pow(-1,i+1) * (float)pow(x,((2*i)-1))/fact((2*i)-1);
   }
   printf("Sum = %f",sum);
}
```

INPUT:
1 5

OUTPUT:

```
Enter x (in radians) : 1
Enter number of terms : 5
Sum = 0.841471
```

CONCLUSION:
Thus, the C program successfully calculates the sum of the sine series for the given number of terms.



Assignment 18

PROBLEM STATEMENT:
Write a C program that accepts a string from the user and performs the following string operations:
i) Calculate length of string
ii) String reversal
iii) Equality check of two strings
iv) Check palindrome
v) Check substring.

OBJECTIVE:
- To understand basic string operations in C.
- To learn how to manipulate strings using loops and functions.
- To perform comparison, reversal, and substring checking.
- To check whether a string is a palindrome.

THEORY:
In C programming, a string is a sequence of characters terminated by a null character ('\0'). Strings are commonly stored in character arrays. Various operations can be performed on strings such as calculating their length, reversing the characters, comparing two strings, checking whether a string is a palindrome, and finding a substring within another string. These operations can be implemented using loops and conditional statements or by using standard library functions from the string.h header file.

PLATFORM:
Linux

CODE:
```
#include <stdio.h>
#include <string.h>

void reverse(int n, char ver[n], char rev[n]){;
    for (int i = 0; i < n-1; i++){
        rev[n - i - 2] = ver[i];
    }
    rev[n-1] = 0;
}

int main() {
    char A[50];
    printf("Enter String A: ");
    fgets(A, 50, stdin);
    char revA[strlen(A)];
    reverse(strlen(A),A,revA);
    char B[50];
```

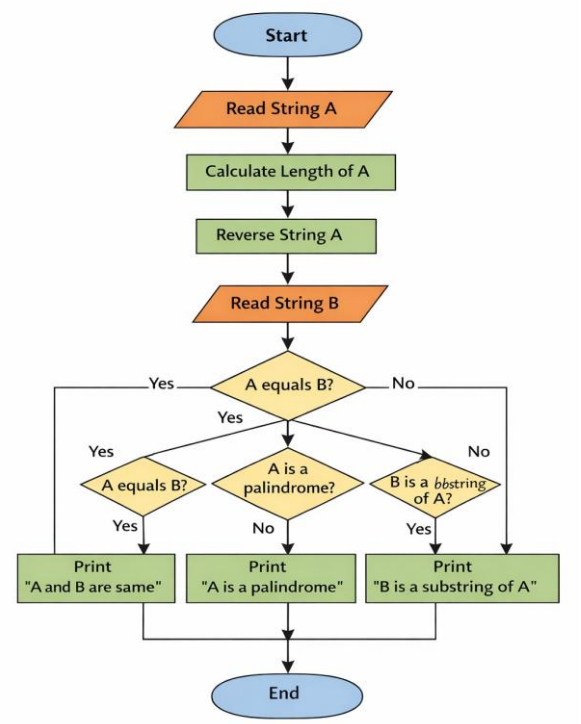
```
    printf("Enter String B: ");
    fgets(B, 50, stdin);
    printf("Length of A = %d\nReversed A is %s\nA and B are %s\nA is %sa palindrome\nB is %sa substring of A",strlen(A),revA,(strcmp(A,B) == 0?"Same":"Different"),(strcmp(A,revA) == 0?"":"not "), (strstr(A,B) != NULL?"":"not "));
}
```

INPUT:
ABC
ABC

OUTPUT:

```
Enter String A: ABC
Enter String B: ABC
Length of A = 4
Reversed A is CBA
A and B are Same
A is not a palindrome
B is a substring of A
```

CONCLUSION:
Thus, the C program successfully performs different string operations such as length calculation, reversal, equality check, palindrome check, and substring detection.



Assignment 19

PROBLEM STATEMENT:
Create Structure EMPLOYEE for storing details (Name, Designation, gender, Date of Joining and Salary). Define function members to compute:
a) total number of employees in an organization
b) count of male and female employee
c) Employee with salary more than 10,000
d) Employee with designation “Asst Manager”

OBJECTIVE:
- To understand the use of structures in C programming.
- To store and manage employee records using structures.
- To perform operations such as counting and filtering employee data

THEORY:
In the C programming language, a structure is a user-defined data type that groups related variables of different data types under a single name. Structures are widely used for representing records such as employee information. In this program, a structure named EMPLOYEE is created to store employee details such as name, designation, gender, date of joining, and salary. By using loops and conditional statements, the program can calculate the total number of employees, count male and female employees, find employees whose salary is greater than 10,000, and display employees having the designation 'Asst Manager'.

PLATFORM:
Linux

CODE:
```
#include <stdio.h>
#include <string.h>

struct employee{
   char name[50];
   char designation[50];
   char gender;
   int date;
   int salary;
};

int main() {
   printf("Enter number of employees: ");
   int n;
   scanf("%d",&n);
   struct employee employees[n];
   int highsalary = 0;
   int male = 0;
   for (int i = 0; i < n; i++){
```

```
        scanf("%c", (char *) stdin);
        printf("Enter name: ");
        fgets(employees[i].name, 50, stdin);
        printf("Enter designation: ");
        fgets(employees[i].designation, 50, stdin);
        printf("Enter gender, date of joining and salary: ");
        scanf("%c %d %d",&employees[i].gender,&employees[i].date, &employees[i].salary);
        if (employees[i].gender == 'm') male++;
        if (employees[i].salary > 10000) highsalary++;
    }
    printf("Male Employees = %d\nFemale Employees = %d\nEmployees with salary over
10000 = %d\nEmployees with designation AsstManager:\n",male,n-male,highsalary);
    for (int i = 0; i < n; i++){
        if (strcmp(employees[i].designation,"AsstManager\n") == 0)
            printf("%s",employees[i].name);
    }
}
```

INPUT:

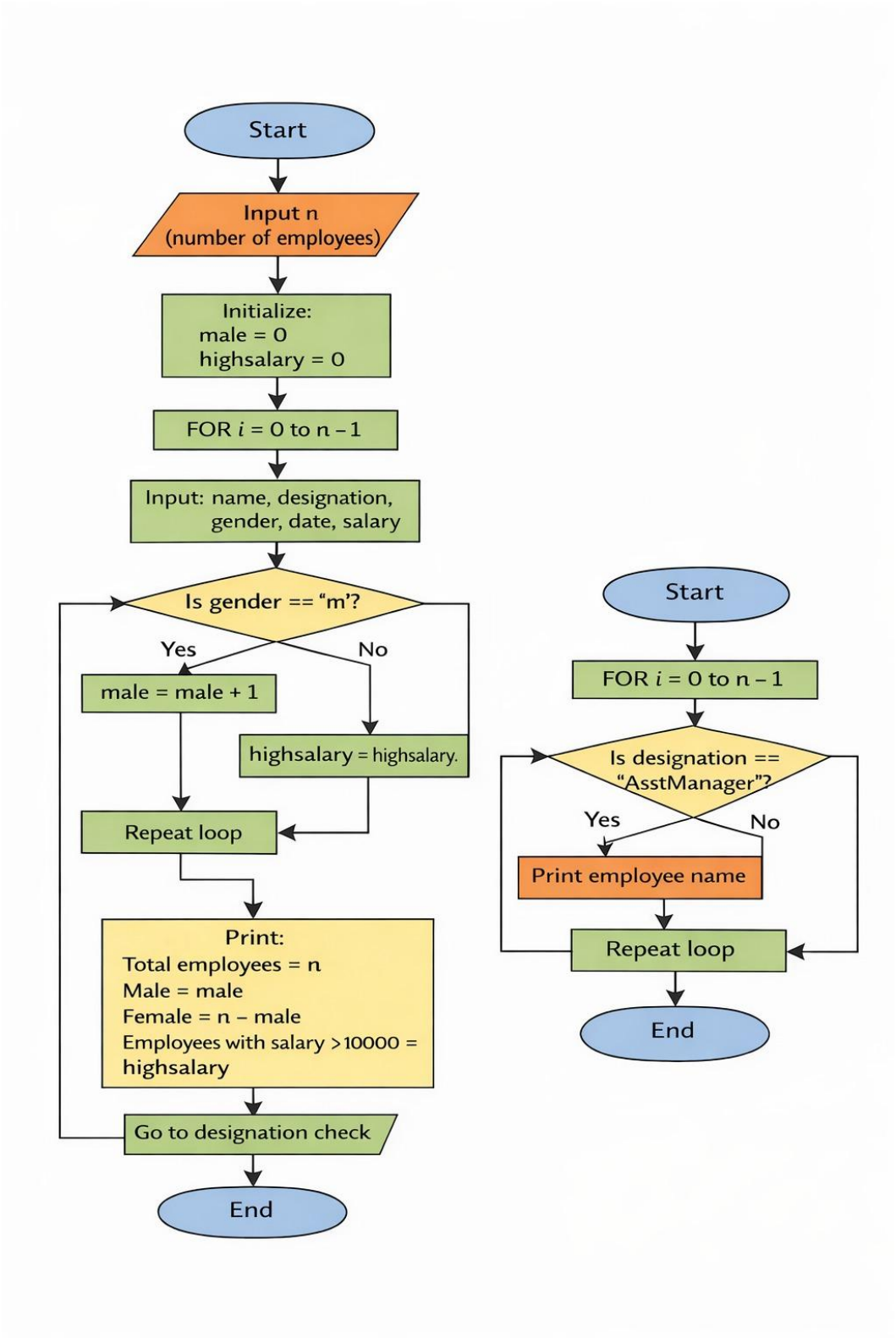
2
Name
AsstManager
m 1 2
Name2
Manager
f 1 2

OUTPUT:

```
Enter number of employees: 2
Enter name: Name
Enter designation: AsstManager
Enter gender, date of joining and salary: m 1 2
Enter name: Name2
Enter designation: Manager
Enter gender, date of joining and salary: f 1 2
Male Employees = 1
Female Employees = 1
Employees with salary over 10000 = 0
Employees with designation AsstManager:
Name
```

CONCLUSION:

Thus, the C program successfully demonstrates the use of structures to store employee details and perform various operations such as counting employees, identifying employees with salary greater than 10,000, and displaying employees with the designation 'Asst Manager'.



Assignment 20

PROBLEM STATEMENT:
Write a C function to swap two numbers with and without using pointers.

OBJECTIVE:
- To understand the concept of function calls in the C programming language.
- To learn the difference between call by value and call by reference.
- To demonstrate swapping of two numbers with and without pointers.

THEORY:
In the C programming language, swapping two numbers can be performed in two ways: without pointers (call by value) and with pointers (call by reference).

When swapping numbers without pointers, the values are passed to the function by value. This means that a copy of the variables is sent to the function, and any changes made inside the function do not affect the original variables.

When swapping numbers with pointers, the addresses of the variables are passed to the function. This allows the function to directly modify the original variables in memory. Pointers therefore enable efficient data manipulation and are widely used in C for handling memory and complex data structures.

PLATFORM:
Linux

CODE:
```
#include <stdio.h>

void swapvar(int a, int b){
   int c = b;
   b = a;
   a = c;
   printf("After swapping without pointers:\na = %d, b = %d\n",a,b);
}

void swappoint(int *a, int *b){
   int d;
   int *c = &d;
   *c = *b;
   *b = *a;
   *a = *c;
   printf("After swapping with pointers:\na = %d, b = %d\n",*a,*b);
}

int main() {
```

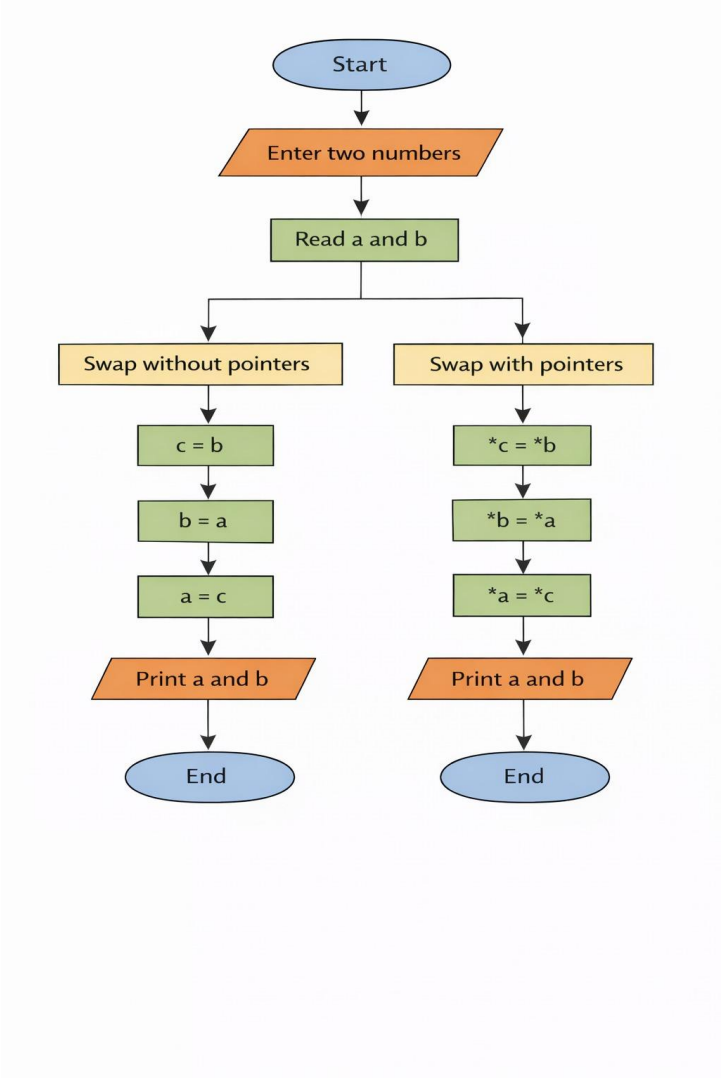
```
    printf("Enter two numbers: ");
    int a,b;
    scanf("%d %d",&a,&b);
    swapvar(a,b);
    swappoint(&a,&b);
}
```

INPUT:
2 5

OUTPUT:

```
Enter two numbers: 2 5
After swapping without pointers:
a = 5, b = 2
After swapping with pointers:
a = 5, b = 2
```

CONCLUSION:
Thus, the C program successfully demonstrates the difference between swapping values using call by value and call by reference. Using pointers allows the function to modify the original variables directly.



Assignment 21

PROBLEM STATEMENT:
Write a C program to copy contents of one file to another using file handling.

OBJECTIVE:
- To understand the concept of file handling in the C programming language.
- To learn how to read data from a file and write data into another file.
- To implement file copy operation using standard file handling functions.

THEORY:
File handling in the C programming language allows programs to store and retrieve data from files. Files are handled using file pointers and standard functions such as fopen(), fclose(), fgetc(), and fputc().

In this program, the source file is opened in read mode, and the destination file is opened in write mode. The program reads characters from the source file one by one using fgetc() and writes them to the destination file using fputc(). This process continues until the end of the file (EOF) is reached. Thus, the contents of the source file are successfully copied into the destination file.

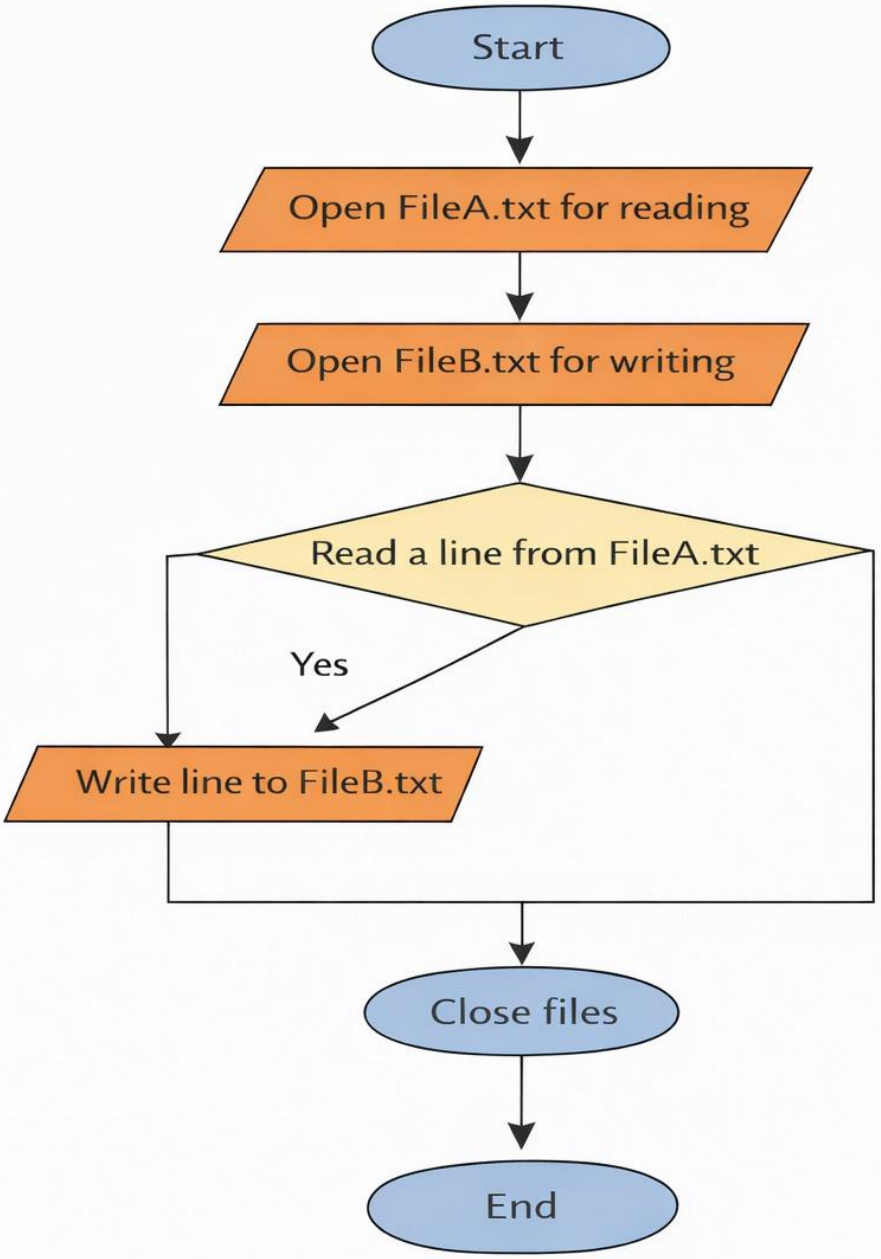
PLATFORM:
Linux

CODE:
```
#include <stdio.h>

int main() {
   FILE *f1 = fopen("FileA.txt","r");
   FILE *f2 = fopen("FileB.txt","w");
   char content[200];
   while (fgets(content, 200, f1) != NULL)
      fprintf(f2,"%s",content);
   fclose(f1);
   fclose(f2);
}
```

INPUT:

OUTPUT:

CONCLUSION:
Thus, the C program successfully demonstrates file handling operations by copying the contents of one file into another using file handling functions.


Start
Open FileA.txt for reading
Open FileB.txt for writing
Read a line from FileA.txt
Yes
Write line to FileB.txt
Close files
End

Assignment 22

PROBLEM STATEMENT:
Write a menu driven program in C to perform various string operations using in-built functions.

OBJECTIVE:
- To understand the concept of strings in the C programming language.
- To learn how to use built-in string functions for string manipulation.
- To perform operations such as length calculation, copying, concatenation, comparison, and reversal using menu options.

.

THEORY:
In the C programming language, a string is a sequence of characters terminated by a null character ('\0'). Strings are stored in character arrays and manipulated using functions available in the string.h header file.
Common built-in string functions include strlen() for finding the length of a string, strcpy() for copying one string to another, strcat() for concatenating two strings, and strcmp() for comparing two strings.
A menu driven program allows the user to select different operations from a list of options. Based on the user's choice, the corresponding string function is executed. This approach makes the program interactive and easy to use.

PLATFORM:
Linux

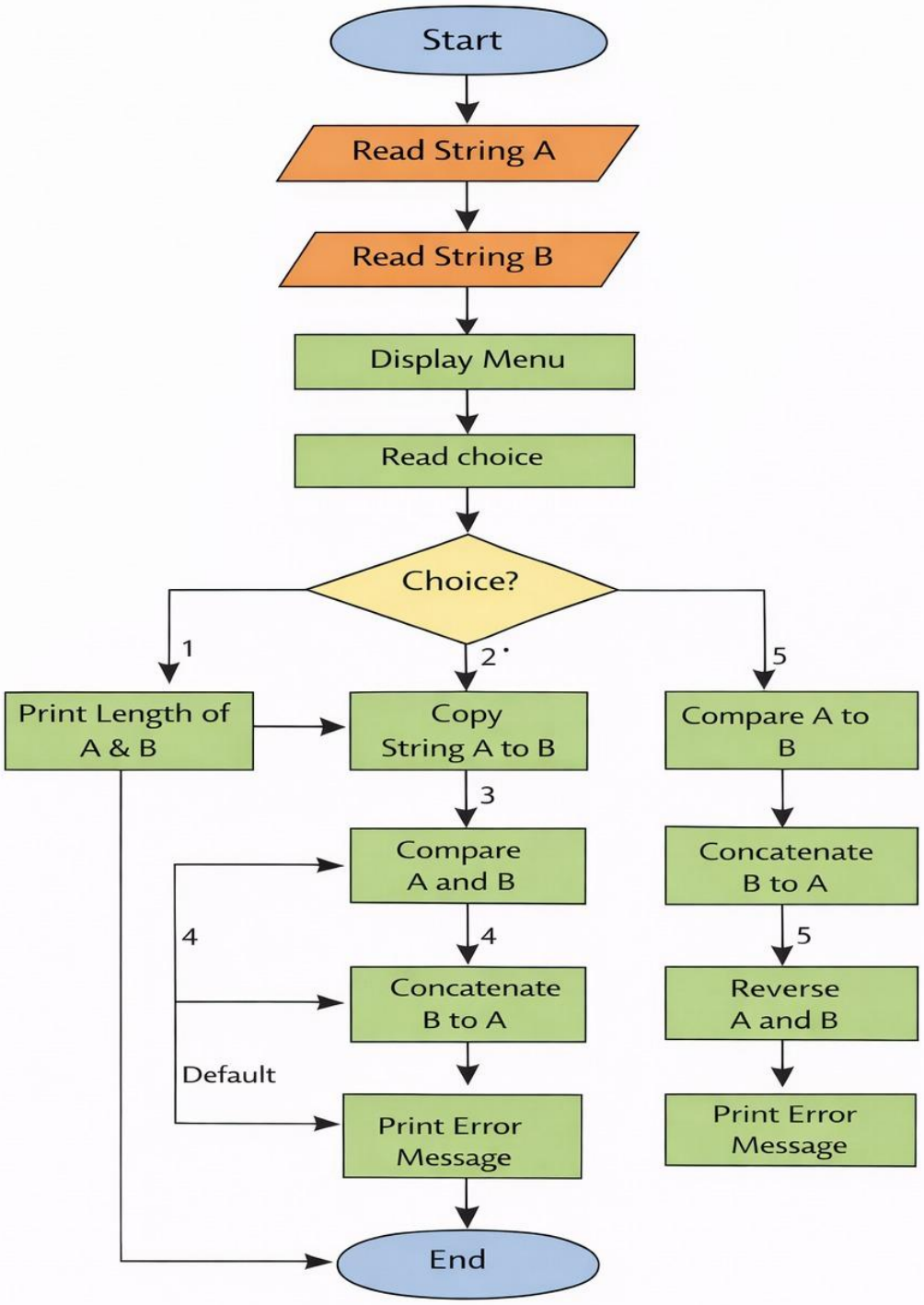
CODE:
```
#include <stdio.h>
#include <string.h>

void reverse(int n, char ver[n], char rev[n]){;
   for (int i = 0; i < n-1; i++){
      rev[n - i - 2] = ver[i];
   }
   rev[n-1] = 0;
}

int main() {
   char A[50];
   printf("Enter String A: ");
   fgets(A, 50, stdin);
   char B[50];
   printf("Enter String B: ");
   fgets(B, 50, stdin);
         char revA[strlen(A)];
         reverse(strlen(A),A,revA);
         char revB[strlen(B)];
```

```
        reverse(strlen(B),B,revB);
    printf("1 - Length\n2 - Copy\n3 - Compare\n4 - Concat\n5 - Reverse\nEnter choice: ");
    int c;
    scanf("%d",&c);
    switch(c){
        case 1:
            printf("Length of A - %d\nLength of B - %d",strlen(A),strlen(B));
            break;
        case 2:
            strcpy(B,A);
            printf("A - %s\nB - %s",A,B);
            break;
        case 3:
            printf("A and B are %s",(strcmp(A,B) == 0?"Same":"Different"));
            break;
        case 4:
            strcat(A,B);
            printf("A - %s",A);
            break;
        case 5:
            printf("A - %s, B - %s",revA,revB);
            break;
        default:
            printf("Error");
    }
}
```

INPUT:
STrING
a
2

OUTPUT:

```
Enter String A: STrING
Enter String B: a
1 - Length
2 - Copy
3 - Compare
4 - Concat
5 - Reverse
Enter choice: 2
A - STrING

B - STrING
```

CONCLUSION:

Thus, the C program successfully demonstrates different string operations using built-in functions through a menu driven approach, making the program interactive and easy to use.